

LeetCode 热题 100

目 录

一. 哈希	5
1. 两数之和 (简单)	5
2. 字母异位词分组 (中等)	5
3. 最长连续序列 (中等)	6
二. 双指针	7
1. 移动零 (简单)	7
2. 盛最多水的容器 (中等)	8
3. 三数之和 (中等)	9
4. 接雨水 (困难)	10
三. 滑动窗口	11
1. 无重复字符的最长子串 (中等)	11
2. 找到字符串中所有字母异位词 (中等)	12
四. 子串	12
1. 和为k的子数组 (中等)	13
2. 滑动窗口最大值 (困难)	13
3. 最小覆盖子串 (困难)	15
五. 普通数组	16
1. 最大子数组和 (中等)	16
2. 合并区间 (中等)	16
3. 轮转数组 (中等)	17
4. 除了自身数组以外的乘积 (中等)	18
5. 缺失的第一个正数 (困难)	19
六. 矩阵	19
1. 矩阵置零 (中等)	20
2. 螺旋矩阵 (中等)	21
3. 旋转图像 (中等)	22
4. 搜索二维矩阵 (中等)	23
七. 链表	23
1. 相交链表 (简单)	24
2. 反转链表 (简单)	25
3. 回文链表 (简单)	26
4. 环形链表 (简单)	27
5. 环形链表II (中等)	27
6. 合并两个有序链表 (简单)	28

示例:	28
7. 两数相加 (中等)	29
8. 删除链表的倒数第N个结点 (中等)	30
9. 两两交换链表中的节点 (中等)	31
10. K个一组翻转链表 (困难)	32
11. 随机链表的复制 (中等)	33
1. 复制节点插入原链表:	34
2. 设置拷贝节点的 random 指针:	34
3. 拆分链表:	34
12. 排序链表 (中等)	35
13. 合并K个升序链表 (困难)	36
2. 两两合并法 (归并): 类似归并排序, 每次两两合并链表, 减少链表数量。时 间复杂度 $O(N \log)$	36
14. LRU缓存 (中等)	37
八. 二叉树	39
1. 二叉树的中序遍历 (简单)	39
2. 二叉树的最大深度 (简单)	40
3. 翻转二叉树 (简单)	41
4. 对称二叉树 (简单)	42
1. 两个节点的值相同	42
2. 左子树的左节点等于右子树的右节点	42
3. 左子树的右节点等于右子树的左节点	42
5. 二叉树的直径 (简单)	43
6. 二叉树的层序遍历 (中等)	44
7. 将有序数组转换为二叉搜索树 (简单)	45
8. 验证二叉搜索树 (中等)	46
9. 二叉搜索树中第K小的元素 (中等)	47
10. 二叉树的右视图 (中等)	48
11. 二叉树展开为链表 (中等)	49
12. 从前序与中序遍历序列构造二叉树 (中等)	50
示例:	50
13. 路径总和III (中等)	52
14. 二叉树的最近公共祖先 (中等)	53
15. 二叉树中的最大路径和 (困难)	54
九. 图论	55

1. 岛屿数量 (中等)	55
2. 腐烂的橘子 (中等)	56
示例:	56
3. 课程表 (中等)	58
4. 实现Trie (前缀树) (中等)	59
十. 回溯	60
1. 全排列 (中等)	60
2. 子集 (中等)	61
3. 电话号码的字母组合 (中等)	62
4. 组合总和 (中等)	63
5. 括号生成 (中等)	64
6. 单词搜索 (中等)	65
7. 分割回文串 (中等)	66
8. N皇后 (困难)	67
十一. 二分查找	68
1. 搜索插入位置 (简单)	68
Python 代码:	68
2. 搜索二维矩阵 (中等)	69
3. 在排序数组中查找元素的第一个和最后一个位置 (中等)	70
4. 搜索旋转排序数组 (中等)	71
5. 寻找旋转排序数组中的最小值 (中等)	72
6. 寻找两个正序数组的中位数 (困难)	73
十二. 栈	74
1. 有效的括号 (简单)	74
2. 最小栈 (中等)	75
3. 字符串解码 (中等)	76
4. 每日温度 (中等)	77
5. 柱状图中最大的矩形 (困难)	77
示例:	77
6. 数组中的第K个最大元素 (中等)	78
7. 前K个高频元素 (中等)	80
8. 数据流的中位数 (困难)	80
十三. 贪心算法	82
1. 买卖股票的最佳时机 (简单)	82
2. 跳跃游戏 (中等)	82

示例:	82
3. 跳跃游戏II (中等)	83
4. 划分字母区间 (中等)	84
十四. 动态规划	85
1. 爬楼梯 (简单)	85
2. 杨辉三角 (简单)	85
3. 打家劫舍 (中等)	86
4. 完全平方数 (中等)	87
5. 零钱兑换 (中等)	88
6. 单词拆分 (中等)	88
7. 最长递增子序列 (中等)	89
Python代码:	89
8. 乘积最大子数组 (中等)	90
9. 分割等和子集 (中等)	91
10. 最长有效括号 (困难)	91
2. 动态规划方法: 定义 $dp[i]$ 表示以索引 i 结尾的最长有效括号长度。	92
十五. 多维动态规划	92
1. 不同路径 (中等)	92
2. 最小路径和 (中等)	93
3. 最长回文子串 (中等)	94
4. 最长公共子序列 (中等)	95
5. 编辑距离 (中等)	96
2. 否则, 可以选择三种操作之一:	96
十六. 技巧	97
1. 只出现一次的数字 (简单)	97
2. 将数组中所有元素依次进行异或操作, 相同的数字会互相抵消为0, 剩下的就是只出现一次的数	97
2. 多数元素 (简单)	97
3. 颜色分类 (中等)	98
4. 下一个排列 (中等)	99
5. 寻找重复数 (中等)	100

一. 哈希

1. 两数之和（简单）

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出 **和为目标值** `target` 的那两个整数，并返回它们的数组下标。你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。你可以按任意顺序返回答案。

示例：

```
输入: nums = [2,7,11,15], target = 9
输出: [0,1]
解释: 因为 nums[0] + nums[1] == 9，返回 [0, 1]。
```

解题思路：本题需要使用 **哈希表（字典）** 来优化查找过程。遍历数组时，对于当前元素 `num`，计算还需要的数 `target - num`，然后在哈希表中查询该数是否已经出现过。如果存在，则说明找到了两个和为 `target` 的数，直接返回它们的下标；如果不存在，则把当前数字及其下标存入哈希表，继续遍历。这样可以把原本需要两层循环的查找优化为一次遍历。

复杂度分析：时间复杂度： $O(n)$ （只遍历一次数组）。空间复杂度： $O(n)$ （需要哈希表存储元素）。

Python代码:

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        hashmap = {}

        for i, num in enumerate(nums):
            complement = target - num

            if complement in hashmap:
                return [hashmap[complement], i]

            hashmap[num] = i
```

2. 字母异位词分组（中等）

给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。

示例：

输入: `strs = ["eat", "tea", "tan", "ate", "nat", "bat"]`

输出: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

解释:

- 在 `strs` 中没有字符串可以通过重新排列来形成 `"bat"`。
- 字符串 `"nat"` 和 `"tan"` 是字母异位词, 因为它们可以重新排列以形成彼此。
- 字符串 `"ate"`, `"eat"` 和 `"tea"` 是字母异位词, 因为它们可以重新排列以形成彼此。

解题思路: 本题需要使用 **哈希表 (字典) + 排序** 来进行分组。字母异位词的特点是 **包含的字母相同, 只是顺序不同**, 因此可以把每个字符串 **排序后的结果作为哈希表的键**。遍历字符串数组时, 将当前字符串排序得到统一的键, 如果该键已经存在于哈希表中, 就把字符串加入对应列表; 如果不存在, 就创建新的列表。遍历结束后, 哈希表中的所有值即为分组结果。

复杂度分析: 时间复杂度: $O(n * k \log k)$, n 是字符串数量, k 是字符串平均长度, 排序每个字符串需要 $k \log k$ 。空间复杂度: $O(n * k)$, 需要存储所有字符串。

Python代码:

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        dict = {}
        for s in strs:
            key = ''.join(sorted(s))
            if key not in dict:
                dict[key] = []
            dict[key].append(s)

        return list(dict.values())
```

3. 最长连续序列 (中等)

给定一个未排序的整数数组 `nums`, 找出数字连续的最长序列 (不要求序列元素在原数组中连续) 的长度。请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例:

输入: `nums = [100,4,200,1,3,2]`

输出: 4

解释: 最长数字连续序列是 `[1, 2, 3, 4]`。它的长度为 4。

解题思路: 本题要求时间复杂度为 $O(n)$, 因此不能先排序 (排序是 $O(n \log n)$)。可以使用 **哈希集合 (set)** 来快速判断某个数字是否存在。具体做法是: 先把所有数字放入集合中, 然后遍历每个数字, 如果当前数字 `num-1` 不在集合中, 说明 `num` 是一个连续序列的起点, 此时不断检查 `num+1`、`num+2...` 是否在集合中, 并统计该连续序列的长度, 更新最长长度即可。由于每个数字最多被访问一次, 因此整体时间复杂度为 $O(n)$ 。

复杂度分析: 时间复杂度: $O(n)$, 每个元素最多被遍历两次 (一次判断起点, 一次扩展序列)。空间复杂度: $O(n)$, 使用集合存储所有元素。

Python代码:

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
```

```

num_set = set(nums)
longest = 0

for num in num_set:
    # 只有当num是序列起点时才开始统计
    if num - 1 not in num_set:
        current = num
        length = 1

        while current + 1 in num_set:
            current += 1
            length += 1

        longest = max(longest, length)

return longest

```

二. 双指针

1. 移动零 (简单)

给定一个数组 `nums`，编写一个函数将所有 `0` 移动到数组的末尾，同时保持非零元素的相对顺序。**请注意**，必须在不复制数组的情况下原地对数组进行操作。

示例：

```

输入：nums = [0,1,0,3,12]
输出：[1,3,12,0,0]

```

解题思路：本题需要使用 **双指针算法** 来原地调整数组顺序。设置一个指针 `slow` 用于指向当前应该放置非零元素的位置，另一个指针 `fast` 用于遍历数组。当 `fast` 遇到非零元素时，就将 `nums[fast]` 与 `nums[slow]` 交换，并将 `slow` 向前移动一位；当遇到 `0` 时只移动 `fast`。这样遍历结束后，所有非零元素都会按原顺序移动到数组前面，而 `0` 会自动被交换到数组末尾，实现原地移动。

复杂度分析：时间复杂度： $O(n)$ （只遍历一次数组）。空间复杂度： $O(1)$ （原地操作，没有额外空间）。

Python代码:

```

class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        slow = 0

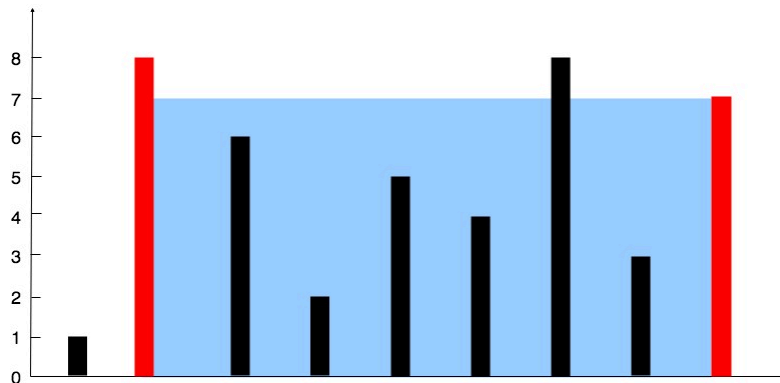
        for fast in range(len(nums)):
            if nums[fast] != 0:
                nums[slow], nums[fast] = nums[fast], nums[slow]
                slow += 1

```

2. 盛最多水的容器（中等）

给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, \text{height}[i])$ 。找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。返回容器可以储存的最大水量。**说明：**你不能倾斜容器。

示例：



输入：[1,8,6,2,5,4,8,3,7]

输出：49

解释：图中垂直线代表输入数组 [1,8,6,2,5,4,8,3,7]。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。

解题思路：本题需要使用 **双指针算法**。初始时分别设置两个指针 `left` 和 `right` 在数组两端，当前容器的面积由 **宽度** ($\text{right} - \text{left}$) $\times$ **两条线中较小的高度** 决定。每次计算当前面积并更新最大值，然后移动 **较短的那条线对应的指针**（因为面积受短板限制，移动较长的一边不会增加高度，只会减少宽度）。不断向中间收缩指针，直到两指针相遇即可找到最大面积。

复杂度分析：时间复杂度： $O(n)$ （左右指针各遍历一次）。空间复杂度： $O(1)$ （只使用常数额外空间）。

Python代码：

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        left, right = 0, len(height) - 1
        res = 0

        while left < right:
            area = min(height[left], height[right]) * (right - left)
            res = max(res, area)

            if height[left] < height[right]:
                left += 1
            else:
                right -= 1

        return res
```


3. 三数之和（中等）

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 `i != j`、`i != k` 且 `j != k`，同时还满足 `nums[i] + nums[j] + nums[k] == 0`。请你返回所有和为 0 且不重复的三元组。**注意：**答案中不可以包含重复的三元组。

示例：

输入：nums = [-1,0,1,2,-1,-4]

输出：[[-1,-1,2],[-1,0,1]]

解释：

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0`。

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0`。

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0`。

不同的三元组是 `[-1,0,1]` 和 `[-1,-1,2]`。

注意，输出的顺序和三元组的顺序并不重要。

解题思路：本题需要使用 **排序 + 双指针算法**。首先对数组进行排序，然后固定一个数 `nums[i]` 作为三元组的第一个元素，问题就转化为在后面的数组中寻找两个数，使得 `nums[left] + nums[right] = -nums[i]`。此时可以使用双指针：`left` 指向 `i+1`，`right` 指向数组末尾，根据三数之和与 0 的大小关系移动指针。同时需要 **跳过重复元素** 来避免出现重复的三元组。遍历所有 `i` 后即可得到所有满足条件的组合。

复杂度分析：时间复杂度： $O(n^2)$ ，排序 $O(n \log n)$ + 双指针遍历 $O(n^2)$ 。空间复杂度： $O(1)$ ，不考虑结果存储。

Python代码：

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        res = []
        n = len(nums)

        for i in range(n - 2):
            # 去重
            if i > 0 and nums[i] == nums[i - 1]:
                continue

            left, right = i + 1, n - 1

            while left < right:
                s = nums[i] + nums[left] + nums[right]

                if s < 0:
                    left += 1
                elif s > 0:
                    right -= 1
                else:
                    res.append([nums[i], nums[left], nums[right]])

                    # 去重
                    while left < right and nums[left] == nums[left + 1]:
                        left += 1
                    while left < right and nums[right] == nums[right - 1]:
```

```
        right -= 1

        left += 1
        right -= 1

    return res
```

4. 接雨水 (困难)

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例：



输入: height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出: 6

解释: 上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

解题思路：本题可以使用 **双指针算法**。设置两个指针 `left` 和 `right` 分别从数组两端向中间移动，同时记录当前左右两侧的最大高度 `left_max` 和 `right_max`。雨水的高度取决于当前位置左右两侧较小的最大高度。当 `left_max < right_max` 时，可以确定 `left` 位置能接的水量为 `left_max - height[left]`（如果为正），然后移动 `left`；否则计算 `right` 位置的水量 `right_max - height[right]`，并移动 `right`。不断向中间收缩指针并累加雨水量即可。

复杂度分析：时间复杂度： $O(n)$ （只遍历数组一次）。空间复杂度： $O(1)$ （只使用常数额外变量）。

Python代码：

```
class Solution:
    def trap(self, height: list[int]) -> int:
        n = len(height)
        if n == 0:
            return 0

        # 1 计算每个位置左边最高
        left_max = [0] * n
        left_max[0] = height[0]
        for i in range(1, n):
            left_max[i] = max(left_max[i - 1], height[i])

        # 2 计算每个位置右边最高
        right_max = [0] * n
        right_max[n - 1] = height[n - 1]
```

```

    for i in range(n - 2, -1, -1):
        right_max[i] = max(right_max[i + 1], height[i])

# 3 逐格算水量
ans = 0
for i in range(n):
    water = min(left_max[i], right_max[i]) - height[i]
    if water > 0:
        ans += water

return ans

```

三. 滑动窗口

1. 无重复字符的最长子串（中等）

给定一个字符串 `s`，请你找出其中不含有重复字符的 **最长子串** 的长度。

示例：

输入：s = "abcabcbb"

输出：3

解释：因为无重复字符的最长子串是 "abc"，所以其长度为 3。注意 "bca" 和 "cab" 也是正确答案。

解题思路： 本题需要使用 **滑动窗口 (Sliding Window)** + **哈希集合**。用两个指针 `left` 和 `right` 表示当前窗口 `[left, right]`，并用集合记录窗口中的字符。`right` 向右扩展窗口，如果当前字符没有出现过，就加入集合并更新最大长度；如果出现重复字符，则不断移动 `left` 并从集合中移除字符，直到窗口重新满足“无重复字符”。在整个过程中维护窗口长度的最大值即可。

复杂度分析： 时间复杂度： $O(n)$ ，每个字符最多进入和移出窗口一次。空间复杂度： $O(\min(n, \text{字符集大小}))$ ，用集合存储窗口字符。

Python代码：

```

class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        char_set = set()
        left = 0
        res = 0

        for right in range(len(s)):
            while s[right] in char_set:
                char_set.remove(s[left])
                left += 1

            char_set.add(s[right])
            res = max(res, right - left + 1)

        return res

```

2. 找到字符串中所有字母异位词（中等）

给定两个字符串 `s` 和 `p`，找到 `s` 中所有 `p` 的 **异位词** 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

示例：

输入：s = "cbaebabacd", p = "abc"

输出：[0,6]

解释：

起始索引等于 0 的子串是 "cba"，它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac"，它是 "abc" 的异位词。

解题思路：本题可以使用 **滑动窗口 + 哈希计数**。先统计字符串 `p` 中每个字符的频次，然后用窗口大小为 `len(p)` 的滑动窗口遍历 `s`。维护窗口内每个字符的频次，如果窗口的字符频次与 `p` 的频次完全一致，则该窗口是一个异位词，记录起始索引。滑动窗口可以用 **左右指针 + 哈希表（字典/数组）** 来维护窗口频次，并在每次右移时更新计数，实现 $O(n)$ 时间复杂度。

复杂度分析：时间复杂度： $O(n)$ ，遍历字符串 `s` 一次，每次更新窗口计数是 $O(1)$ （字符固定为26个小写字母）。空间复杂度： $O(1)$ ，只使用长度为26的数组存储频次。

Python代码：

```
from collections import Counter

class Solution:
    def findAnagrams(self, s: str, p: str) -> List[int]:
        res = []
        len_p = len(p)
        counter_p = Counter(p)
        window = Counter()

        for i, char in enumerate(s):
            window[char] += 1

            if i >= len_p:
                left_char = s[i - len_p]
                if window[left_char] == 1:
                    del window[left_char]
                else:
                    window[left_char] -= 1

            if window == counter_p:
                res.append(i - len_p + 1)

        return res
```

四. 子串

1. 和为k的子数组（中等）

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的子数组的个数。子数组是数组中元素的连续非空序列。

示例：

```
输入: nums = [1,1,1], k = 2
输出: 2
```

解题思路：本题可以使用 **前缀和 + 哈希表** 来优化查找。核心思想是：如果当前前缀和为 `curr_sum`，并且存在一个之前的前缀和 `curr_sum - k`，那么说明从该前缀和之后到当前索引的子数组和为 `k`。遍历数组时维护一个哈希表 `prefix_count`，记录每个前缀和出现的次数，每遇到一个新元素就更新 `curr_sum` 并查找 `curr_sum - k` 是否在哈希表中，如果存在则累加结果。这样可以在一次遍历中统计所有符合条件的子数组。

复杂度分析：时间复杂度： $O(n)$ ，遍历一次数组，每次哈希表操作为 $O(1)$ 。空间复杂度： $O(n)$ ，哈希表存储前缀和及出现次数。

Python代码:

```
from collections import defaultdict

class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        prefix_count = defaultdict(int)
        prefix_count[0] = 1 # 初始前缀和为0出现一次
        curr_sum = 0
        res = 0

        for num in nums:
            curr_sum += num
            res += prefix_count.get(curr_sum - k, 0)
            prefix_count[curr_sum] += 1

        return res
```

2. 滑动窗口最大值（困难）

给你一个整数数组 `nums`，有一个大小为 `k` 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 `k` 个数字。滑动窗口每次只向右移动一位。返回 滑动窗口中的最大值。

示例：

输入: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

输出: `[3,3,5,5,6,7]`

解释:

滑动窗口的位置	最大值
-----	-----
[1 3 -1] -3 5 3 6 7	3
1 [3 -1 -3] 5 3 6 7	3
1 3 [-1 -3 5] 3 6 7	5
1 3 -1 [-3 5 3] 6 7	5
1 3 -1 -3 [5 3 6] 7	6
1 3 -1 -3 5 [3 6 7]	7

解题思路: 本题可以使用 **单调双端队列 (Deque)** 来在 $O(n)$ 时间内求解。维护一个双端队列 `dq`, 存储当前窗口内可能成为最大值的元素 **下标**, 保证队列从头到尾元素对应的值单调递减。遍历数组时:

- 1 移除队列头部已经滑出窗口的下标。
- 2 移除队列尾部小于当前元素的下标 (因为它们不可能成为窗口最大值)。
- 3 将当前元素下标加入队列尾。
- 4 当窗口形成时 (`i >= k - 1`), 队列头部就是当前窗口的最大值。

这种方法保证每个元素最多进队和出队一次, 实现 $O(n)$ 时间复杂度。

复杂度分析: 时间复杂度: $O(n)$, 每个元素最多进出队列一次。空间复杂度: $O(k)$, 队列最多存储 `k` 个元素下标。

Python代码:

```
from collections import deque

class Solution:
    def maxSlidingWindow(self, nums: List[int], k: int) -> List[int]:
        if not nums:
            return []

        dq = deque()
        res = []

        for i, num in enumerate(nums):
            # 移除队首已滑出窗口的下标
            while dq and dq[0] < i - k + 1:
                dq.popleft()

            # 移除队尾小于当前元素的下标
            while dq and nums[dq[-1]] < num:
                dq.pop()

            dq.append(i)

            # 当窗口形成时, 队首就是最大值
            if i >= k - 1:
                res.append(nums[dq[0]])

        return res
```

3. 最小覆盖子串 (困难)

给定两个字符串 `s` 和 `t`，长度分别是 `m` 和 `n`，返回 `s` 中的 **最短窗口子串**，使得该子串包含 `t` 中的每一个字符（包括重复字符）。如果没有这样的子串，返回空字符串 `""`。测试用例保证答案唯一。

示例：

```
输入: s = "ADOBECODEBANC", t = "ABC"
输出: "BANC"
解释: 最小覆盖子串 "BANC" 包含来自字符串 t 的 'A'、'B' 和 'C'。
```

解题思路：本题需要使用 **滑动窗口 + 哈希表计数**。先统计字符串 `t` 中每个字符需要的数量，然后用两个指针 `left` 和 `right` 形成滑动窗口遍历 `s`。当 `right` 扩展窗口时更新窗口内字符计数，当窗口已经包含 `t` 的所有字符（即满足需求字符数量）时，就尝试移动 `left` 收缩窗口，并更新当前最短子串长度。整个过程中维护一个变量记录满足条件的最小窗口范围，最终返回该子串。核心思想：先扩张窗口满足条件 → 再收缩窗口寻找最小长度。

复杂度分析：时间复杂度： $O(m + n)$ ，每个字符最多被左右指针访问一次。空间复杂度： $O(\text{字符集大小})$ ，哈希表存储字符频次。

Python代码：

```
from collections import Counter

class Solution:
    def minWindow(self, s: str, t: str) -> str:
        need = Counter(t)
        window = {}
        left = 0
        valid = 0

        start = 0
        min_len = float('inf')

        for right in range(len(s)):
            c = s[right]
            if c in need:
                window[c] = window.get(c, 0) + 1
                if window[c] == need[c]:
                    valid += 1

            while valid == len(need):
                if right - left + 1 < min_len:
                    start = left
                    min_len = right - left + 1

                d = s[left]
                left += 1

                if d in need:
                    if window[d] == need[d]:
                        valid -= 1
                    window[d] -= 1
```

```
return "" if min_len == float('inf') else s[start:start + min_len]
```

五. 普通数组

1. 最大子数组和（中等）

给你一个整数数组 `nums`，请你找出一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。**子数组**是数组中的一个连续部分。

示例：

输入: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
输出: 6
解释: 连续子数组 `[4,-1,2,1]` 的和最大，为 6。

解题思路：本题需要用到**动态规划（Kadane算法）**。核心思想是：遍历数组时，维护一个当前子数组的最大和 `cur_sum`。如果当前累计和为负数，那么继续加下一个数只会让结果更小，因此应当从当前元素重新开始计算子数组。每一步都更新全局最大值 `max_sum`。最终遍历结束即可得到最大子数组和。该方法只需一次遍历即可解决问题。

复杂度分析：时间复杂度：O(n)，只遍历数组一次。空间复杂度：O(1)，只使用常数级变量。

Python代码:

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        cur_sum = nums[0]
        max_sum = nums[0]

        for i in range(1, len(nums)):
            cur_sum = max(nums[i], cur_sum + nums[i])
            max_sum = max(max_sum, cur_sum)

        return max_sum
```

2. 合并区间（中等）

以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

示例：

输入: `intervals = [[1,3],[2,6],[8,10],[15,18]]`
输出: `[[1,6],[8,10],[15,18]]`
解释: 区间 `[1,3]` 和 `[2,6]` 重叠，将它们合并为 `[1,6]`。

解题思路：本题主要用到 **排序 + 贪心算法**。首先按照每个区间的起点 `start` 对区间进行排序，然后遍历排序后的区间集合，维护一个当前正在合并的区间 `current`。如果下一个区间的起点 `next_start` 小于等于 `current` 的终点 `current_end`，说明存在重叠，需要将当前区间终点更新为 `max(current_end, next_end)`。否则，说明没有重叠，将 `current` 加入结果，并把下一个区间作为新的 `current`。遍历完成后，结果数组即为合并后的不重叠区间集合。

复杂度分析：时间复杂度： $O(n \log n)$ ，排序占主导，其余遍历 $O(n)$ 。空间复杂度： $O(n)$ ，用于存储合并后的区间数组。

Python代码：

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        if not intervals:
            return []

        # 按起点排序
        intervals.sort(key=lambda x: x[0])
        merged = [intervals[0]]

        for i in range(1, len(intervals)):
            current = merged[-1]
            next_interval = intervals[i]

            if next_interval[0] <= current[1]:
                # 存在重叠，合并
                current[1] = max(current[1], next_interval[1])
            else:
                # 不重叠，直接加入结果
                merged.append(next_interval)

        return merged
```

3. 轮转数组 (中等)

给定一个整数数组 `nums`，将数组中的元素向右轮转 `k` 个位置，其中 `k` 是非负数。

示例：

```
输入: nums = [1,2,3,4,5,6,7], k = 3
输出: [5,6,7,1,2,3,4]
解释:
向右轮转 1 步: [7,1,2,3,4,5,6]
向右轮转 2 步: [6,7,1,2,3,4,5]
向右轮转 3 步: [5,6,7,1,2,3,4]
```

解题思路：本题可以使用 **数组反转技巧** 来在原地实现右旋转。核心步骤如下：先对整个数组进行 **整体反转**。再对前 `k` 个元素反转。最后对剩下的 `n-k` 个元素反转。这样最终数组就实现了向右轮转 `k` 个位置。注意要对 `k` 取模 `n`，防止 `k` 大于数组长度。

复杂度分析：时间复杂度： $O(n)$ ，每个元素最多交换一次。空间复杂度： $O(1)$ ，原地操作，没有额外数组。

Python代码:

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        n = len(nums)
        k %= n # 防止 k > n

        def reverse(left, right):
            while left < right:
                nums[left], nums[right] = nums[right], nums[left]
                left += 1
                right -= 1

        # 1. 整体反转
        reverse(0, n - 1)
        # 2. 前 k 个元素反转
        reverse(0, k - 1)
        # 3. 剩下元素反转
        reverse(k, n - 1)
```

4. 除了自身数组以外的乘积（中等）

给你一个整数数组 `nums`，返回 数组 `answer`，其中 `answer[i]` 等于 `nums` 中除了 `nums[i]` 之外其余各元素的乘积。题目数据 **保证** 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 **32 位** 整数范围内。请 **不要使用除法**，且在 `O(n)` 时间复杂度内完成此题。

示例：

```
输入：nums = [1,2,3,4]
输出：[24,12,8,6]
```

解题思路： 本题可以使用 **前缀乘积 + 后缀乘积** 来在 $O(n)$ 时间复杂度和 $O(1)$ 除结果数组外的空间内完成。核心步骤：创建一个数组 `answer`，其中 `answer[i]` 存储 **索引 i 左侧元素的乘积**。通过一个变量 `R` 从右向左遍历，累乘 **索引 i 右侧元素的乘积**，同时乘到 `answer[i]` 上。最终 `answer[i]` 就是除自身以外的乘积。

复杂度分析： 时间复杂度： $O(n)$ ，遍历两次数组即可。空间复杂度： $O(1)$ ，不计算返回数组之外的额外空间。

Python代码:

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        answer = [1] * n

        # 先计算左侧乘积
        left_prod = 1
        for i in range(n):
            answer[i] = left_prod
            left_prod *= nums[i]
```

```

# 再计算右侧乘积并更新答案
right_prod = 1
for i in reversed(range(n)):
    answer[i] *= right_prod
    right_prod *= nums[i]

return answer

```

5. 缺失的第一个正数（困难）

给你一个未排序的整数数组 `nums`，请你找出其中没有出现的最小的正整数。请你实现时间复杂度为 $O(n)$ 并且只使用常数级别额外空间的解决方案。

示例：

输入：nums = [1,2,0]
 输出：3
 解释：范围 [1,2] 中的数字都在数组中。

解题思路：本题可以使用 **原地哈希（置换数组）** 技巧，在 $O(n)$ 时间复杂度和 $O(1)$ 空间下完成。核心思想是：遍历数组，将每个正整数 `x` 放到下标 `x-1` 位置上，例如 `1` 放到索引 `0`，`2` 放到索引 `1`，以此类推。只交换数组内 **1 到 n 的正整数**，忽略非正数和大于 `n` 的数。完成置换后，再次遍历数组，找到第一个不满足 `nums[i] == i + 1` 的位置，返回 `i + 1`。如果所有位置都满足，说明数组包含 `[1,n]`，缺失的最小正数是 `n+1`。

复杂度分析：时间复杂度： $O(n)$ ，每个元素最多交换一次。空间复杂度： $O(1)$ ，原地操作，没有额外数组。

Python代码：

```

class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        n = len(nums)
        for i in range(n):
            while 1 <= nums[i] <= n and nums[nums[i]-1] != nums[i]:
                # 将 nums[i] 放到正确位置 nums[i]-1
                nums[nums[i]-1], nums[i] = nums[i], nums[nums[i]-1]

        # 检查第一个不满足 nums[i] == i+1 的位置
        for i in range(n):
            if nums[i] != i + 1:
                return i + 1

        return n + 1

```

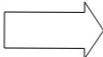
六. 矩阵

1. 矩阵置零 (中等)

给定一个 $m \times n$ 的矩阵，如果一个元素为 0，则将其所在行和列的所有元素都设为 0。请使用 [原地](#) 算法。

示例 1:

1	1	1
1	0	1
1	1	1



1	0	1
0	0	0
1	0	1

输入: matrix = [[1,1,1],[1,0,1],[1,1,1]]

输出: [[1,0,1],[0,0,0],[1,0,1]]

解题思路: 本题需要使用 **原地标记技巧**。直接把行列置零可能会影响后续判断，因此可以使用矩阵的 **第一行和第一列** 作为标记:

- 1 遍历整个矩阵，如果 `matrix[i][j] == 0`，就把对应行的第一列 `matrix[i][0]` 和对应列的第一行 `matrix[0][j]` 置为 0。
- 2 用两个额外变量 `first_row_zero` 和 `first_col_zero` 记录原始第一行和第一列是否含 0，以免被标记覆盖。
- 3 再次遍历矩阵（跳过第一行和第一列），根据标记置零。
- 4 最后根据 `first_row_zero` 和 `first_col_zero` 决定是否把第一行和第一列置零。

复杂度分析: 时间复杂度: $O(m * n)$ ，遍历矩阵两次。空间复杂度: $O(1)$ ，只使用了两个额外变量标记第一行和第一列。

Python代码:

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        m, n = len(matrix), len(matrix[0])
        first_row_zero = any(matrix[0][j] == 0 for j in range(n))
        first_col_zero = any(matrix[i][0] == 0 for i in range(m))

        # 用第一行和第一列标记需要置零的行列
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][j] == 0:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        # 根据标记置零
        for i in range(1, m):
            for j in range(1, n):
                if matrix[i][0] == 0 or matrix[0][j] == 0:
                    matrix[i][j] = 0

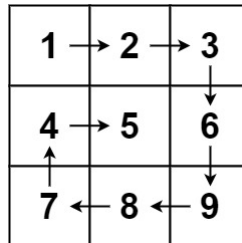
        # 处理第一行
        if first_row_zero:
            for j in range(n):
                matrix[0][j] = 0
```

```
# 处理第一列
if first_col_zero:
    for i in range(m):
        matrix[i][0] = 0
```

2. 螺旋矩阵（中等）

给你一个 m 行 n 列的矩阵 `matrix`，请按照 **顺时针螺旋顺序**，返回矩阵中的所有元素。

示例：



输入: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
 输出: `[1,2,3,6,9,8,7,4,5]`

解题思路：本题需要使用 **模拟 + 边界控制**。通过维护四个边界 `top`、`bottom`、`left`、`right` 来表示当前未遍历的矩阵范围，然后按照 **顺时针四个方向依次遍历：右 → 下 → 左 → 上**。每遍历完一个方向就收缩对应边界（如遍历完上边界后 `top += 1`）。在遍历过程中需要判断边界是否仍然有效，防止重复遍历。当 `top > bottom` 或 `left > right` 时结束循环。

复杂度分析：时间复杂度： $O(m \times n)$ ，每个元素访问一次。空间复杂度： $O(1)$ （不算返回结果）。

Python代码：

```
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        res = []
        top, bottom = 0, len(matrix) - 1
        left, right = 0, len(matrix[0]) - 1

        while top <= bottom and left <= right:

            # 从左到右
            for j in range(left, right + 1):
                res.append(matrix[top][j])
            top += 1

            # 从上到下
            for i in range(top, bottom + 1):
                res.append(matrix[i][right])
            right -= 1

            if top <= bottom:
                # 从右到左
                for j in range(right, left - 1, -1):
                    res.append(matrix[bottom][j])
```

```

        bottom -= 1

    if left <= right:
        # 从下到上
        for i in range(bottom, top - 1, -1):
            res.append(matrix[i][left])
        left += 1

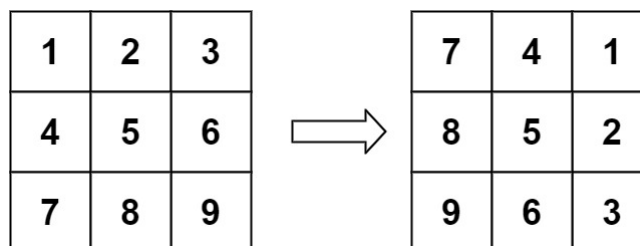
    return res

```

3. 旋转图像（中等）

给定一个 $n \times n$ 的二维矩阵 `matrix` 表示一个图像。请你将图像顺时针旋转 90 度。你必须在原地旋转图像，这意味着你需要直接修改输入的二维矩阵。**请不要** 使用另一个矩阵来旋转图像。

示例：



输入: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`

输出: `[[7,4,1],[8,5,2],[9,6,3]]`

解题思路：本题需要使用 **矩阵变换 + 原地操作**。顺时针旋转 90° 可以拆成两个步骤完成：**1 先转置矩阵 (transpose)**：把 `matrix[i][j]` 和 `matrix[j][i]` 交换，相当于沿 **主对角线翻转**。**2 再翻转每一行 (reverse)**：把每一行左右反转，即可得到顺时针旋转 90° 的结果。这样可以在 **原地完成旋转**，不需要额外矩阵。

复杂度分析：时间复杂度： $O(n^2)$ 。空间复杂度： $O(1)$ （原地操作）。

Python代码：

```

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)

        # 1. 转置矩阵
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        # 2. 每一行反转
        for row in matrix:
            row.reverse()

```

4. 搜索二维矩阵（中等）

编写一个高效的算法来搜索 $m \times n$ 矩阵 `matrix` 中的一个目标值 `target`。该矩阵具有以下特性：每行的元素从左到右升序排列。每列的元素从上到下升序排列。

示例：

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

```
输入: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],
[18,21,23,26,30]], target = 5
输出: true
```

解题思路： 本题利用矩阵的 **行递增 + 列递增性质**。可以从 **右上角** 开始搜索：如果当前元素 `matrix[i][j] > target`，说明这一列下面的数都会更大，因此 **向左移动** `j--`。如果当前元素 `matrix[i][j] < target`，说明这一行左边的数都更小，因此 **向下移动** `i++`。如果相等，则找到目标返回 `True`。因为每一步都会排除一整行或一整列，所以时间复杂度是 $O(m + n)$ 。

复杂度分析： 时间复杂度： $O(m + n)$ ，最坏情况最多移动 $m + n$ 次。空间复杂度： $O(1)$ ，只使用常数变量。

Python代码：

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m, n = len(matrix), len(matrix[0])

        i, j = 0, n - 1 # 从右上角开始

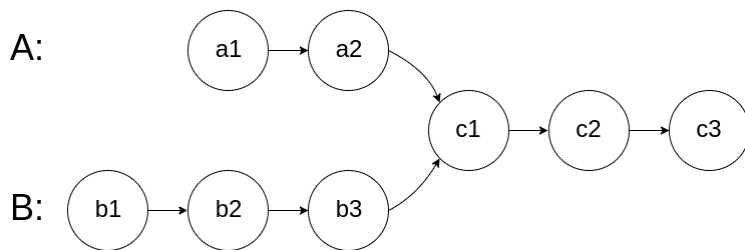
        while i < m and j >= 0:
            if matrix[i][j] == target:
                return True
            elif matrix[i][j] > target:
                j -= 1
            else:
                i += 1

        return False
```

七. 链表

1. 相交链表（简单）

给你两个单链表的头节点 `headA` 和 `headB`，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 `null`。图示两个链表在节点 `c1` 开始相交：



题目数据 **保证** 整个链式结构中不存在环。**注意**，函数返回结果后，链表必须 **保持其原始结构**。

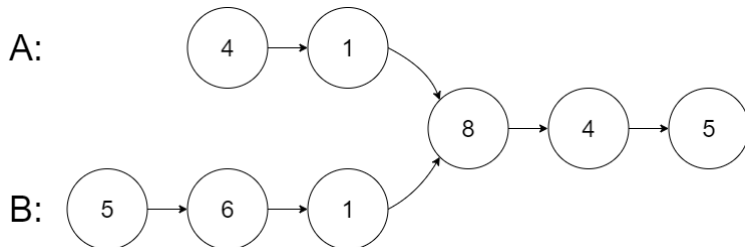
自定义评测：

评测系统 的输入如下（你设计的程序 **不适用** 此输入）：

- `intersectVal` - 相交的起始节点的值。如果不存在相交节点，这一值为 `0`
- `listA` - 第一个链表
- `listB` - 第二个链表
- `skipA` - 在 `listA` 中（从头节点开始）跳到交叉节点的节点数
- `skipB` - 在 `listB` 中（从头节点开始）跳到交叉节点的节点数

评测系统将根据这些输入创建链式数据结构，并将两个头节点 `headA` 和 `headB` 传递给你的程序。如果程序能够正确返回相交节点，那么你的解决方案将被 **视作正确答案**。

示例：



输入: `intersectVal = 8`, `listA = [4,1,8,4,5]`, `listB = [5,6,1,8,4,5]`, `skipA = 2`, `skipB = 3`

输出: `Intersected at '8'`

解释: 相交节点的值 **为 8**（注意，如果两个链表相交则不能为 `0`）。

从各自的表头开始算起，链表 A 为 `[4,1,8,4,5]`，链表 B 为 `[5,6,1,8,4,5]`。

在 A 中，相交节点前有 2 个节点；在 B 中，相交节点前有 3 个节点。

– 请注意相交节点的值不为 `1`，因为在链表 A 和链表 B 之中值为 `1` 的节点（A 中第二个节点和 B 中第三个节点）是不同的节点。换句话说，它们在内存中指向两个不同的位置，而链表 A 和链表 B 中值为 `8` 的节点（A 中第三个节点，B 中第四个节点）在内存中指向相同的位置。

解题思路： 本题需要使用 **双指针技巧**。分别用两个指针 `pA` 和 `pB` 从 `headA` 和 `headB` 出发遍历链表。当一个指针走到链表末尾时，就让它跳到另一个链表的头部继续走。这样两个指针最终会走过 **相同的总长度 (A+B)**，如果存在相交节点，它们会在相交节点处相遇；如果不存在相交节点，两个指针最终都会变成 `None`。该方法不需要额外空间，并且不会改变链表结构。

复杂度分析： 时间复杂度： $O(m + n)$ ，两个链表最多各遍历一次。空间复杂度： $O(1)$ ，只使用两个指针。

Python 代码：


```
class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> Optional[ListNode]:
        pA, pB = headA, headB

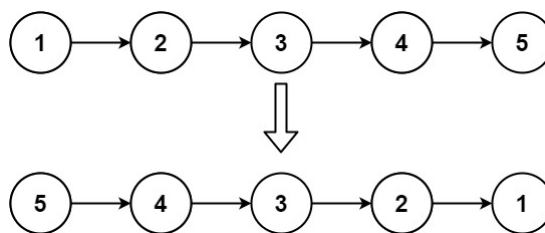
        while pA != pB:
            pA = pA.next if pA else headB
            pB = pB.next if pB else headA

        return pA
```

2. 反转链表（简单）

给你单链表的头节点 `head`，请你反转链表，并返回反转后的链表。

示例：



输入：head = [1,2,3,4,5]

输出：[5,4,3,2,1]

解题思路： 本题使用 **链表迭代反转法（双指针）**。遍历链表时，每次把当前节点 `cur` 的 `next` 指针指向前一个节点 `prev`，从而实现链表方向反转。过程中需要先保存原本的 `next` 节点，否则会丢失后续链表。遍历结束后，`prev` 就是新的头节点。反转过程本质就是：`cur.next = prev` 然后三个指针整体向前移动。

复杂度分析： 时间复杂度： $O(n)$ ，遍历链表一次。空间复杂度： $O(1)$ ，只使用常数指针。

Python代码：

```
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev = None
        cur = head

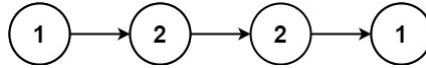
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt

        return prev
```

3. 回文链表（简单）

给你一个单链表的头节点 `head`，请你判断该链表是否为回文链表。如果是，返回 `true`；否则，返回 `false`。

示例：



输入: `head = [1,2,2,1]`

输出: `true`

解题思路：本题需要判断链表是否为回文结构，可以通过 **快慢指针 + 反转链表** 来解决。首先使用快慢指针找到链表的中点（快指针每次走两步，慢指针走一步）。当快指针到达末尾时，慢指针正好在链表中间。然后将 **后半部分链表进行反转**，再从头节点和反转后的后半部分同时向后遍历，逐个比较节点值。如果全部相同则是回文链表，否则不是。

复杂度分析：该方法时间复杂度为 $O(n)$ ，空间复杂度 $O(1)$ 。

Python代码：

```
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        slow = fast = head

        # 找中点
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        # 反转后半部分
        prev = None
        cur = slow
        while cur:
            nxt = cur.next
            cur.next = prev
            prev = cur
            cur = nxt

        # 比较前后两部分
        left = head
        right = prev

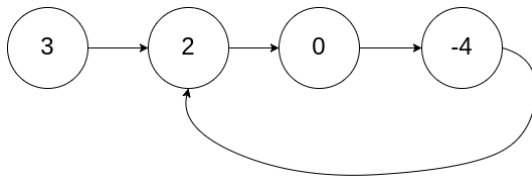
        while right:
            if left.val != right.val:
                return False
            left = left.next
            right = right.next

        return True
```

4. 环形链表（简单）

给你一个链表的头节点 `head`，判断链表中是否有环。如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（索引从 0 开始）。**注意：**`pos` **不作为参数进行传递**。仅仅是为了标识链表的实际情况。*如果链表中存在环，则返回 `true`。否则，返回 `false`。*

示例：



输入：head = [3,2,0,-4], pos = 1

输出：true

解释：链表中有一个环，其尾部连接到第二个节点。

解题思路：本题可以使用 **快慢指针（Floyd 判圈算法）**。定义两个指针 `slow` 和 `fast`，`slow` 每次移动一步，`fast` 每次移动两步。如果链表中存在环，那么快指针最终一定会追上慢指针（两者相遇）；如果链表没有环，则快指针会先到达 `None`。因此在遍历过程中只要出现 `slow == fast` 就说明存在环。这种方法只需要遍历一次链表即可完成检测。

复杂度分析：该方法时间复杂度为 $O(n)$ ，空间复杂度 $O(1)$ 。

Python代码：

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = head
        fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

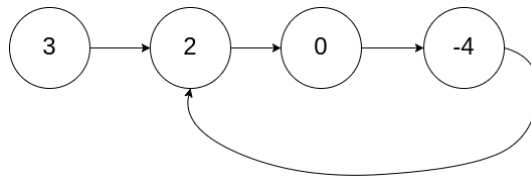
            if slow == fast:
                return True

        return False
```

5. 环形链表II（中等）

给定一个链表的头节点 `head`，返回链表开始入环的第一个节点。如果链表无环，则返回 `null`。如果链表中有某个节点，可以通过连续跟踪 `next` 指针再次到达，则链表中存在环。为了表示给定链表中的环，评测系统内部使用整数 `pos` 来表示链表尾连接到链表中的位置（索引从 0 开始）。如果 `pos` 是 `-1`，则在该链表中没有环。**注意：**`pos` **不作为参数进行传递**，仅仅是为了标识链表的实际情况。**不允许修改** 链表。

示例：



输入: head = [3,2,0,-4], pos = 1

输出: 返回索引为 1 的链表节点

解释: 链表中有一个环, 其尾部连接到第二个节点。

解题思路: 本题仍然使用 **快慢指针 (Floyd 判圈算法)**。首先使用快慢指针判断链表是否有环: **slow** 每次走一步, **fast** 每次走两步, 如果两者相遇说明存在环。设相遇点为 **meet**。此时将一个指针重新放回链表头节点 **head**, 另一个指针保持在 **meet** 位置, 然后两个指针每次都走一步, 它们再次相遇的位置就是 **环的入口节点**。原因是: 从链表头到环入口的距离等于从相遇点到环入口的距离 (数学推导来自 Floyd 算法性质)。

复杂度分析: 该方法时间复杂度为 $O(n)$, 空间复杂度 $O(1)$ 。

Python代码:

```

class Solution:
    def detectCycle(self, head: Optional[ListNode]) -> Optional[ListNode]:
        slow = fast = head

        # 判断是否有环
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

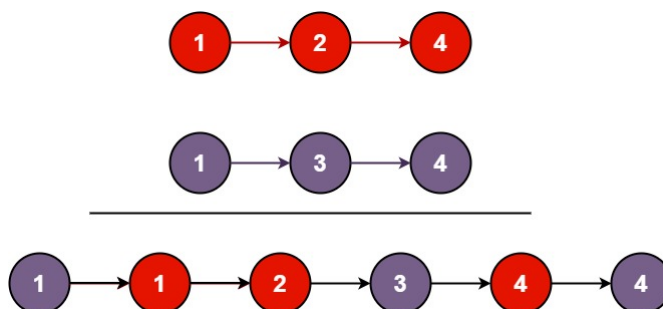
        if slow == fast:
            # 找到环入口
            p = head
            while p != slow:
                p = p.next
                slow = slow.next
            return p

        return None
  
```

6. 合并两个有序链表 (简单)

将两个升序链表合并为一个新的 **升序** 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例:



输入: l1 = [1,2,4], l2 = [1,3,4]
输出: [1,1,2,3,4,4]

解题思路：本题可以使用 **双指针 + 迭代** 的方法。由于两个链表已经是升序排列，因此可以同时遍历两个链表，比较当前节点的值，将较小的节点接到新链表后面，并移动对应指针。为了方便操作，可以创建一个 **虚拟头节点 (dummy)**，用一个 **cur** 指针不断向后连接节点。当其中一个链表遍历结束后，直接将另一个链表剩余部分接到新链表末尾即可。

复杂度分析：该方法时间复杂度为 $O(m+n)$ ，空间复杂度 $O(1)$ 。

Python代码：

```
class Solution:
    def mergeTwoLists(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0)
        cur = dummy

        while l1 and l2:
            if l1.val <= l2.val:
                cur.next = l1
                l1 = l1.next
            else:
                cur.next = l2
                l2 = l2.next
            cur = cur.next

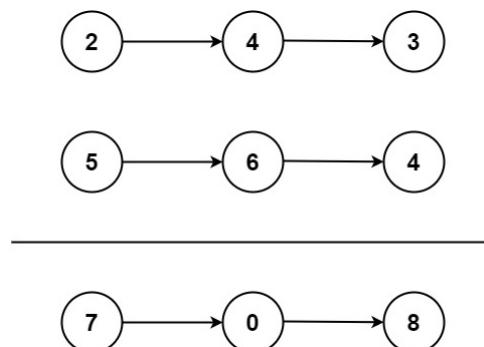
        cur.next = l1 if l1 else l2

        return dummy.next
```

7. 两数相加（中等）

给你两个 **非空** 的链表，表示两个非负的整数。它们每位数字都是按照 **逆序** 的方式存储的，并且每个节点只能存储 **一位** 数字。请你将两个数相加，并以相同形式返回一个表示和的链表。你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

示例：



输入: l1 = [2,4,3], l2 = [5,6,4]
输出: [7,0,8]
解释: 342 + 465 = 807.

解题思路： 本题本质是 **模拟加法运算 + 链表遍历**。因为数字是 **逆序存储**，所以链表的头节点就是个位，正好符合从低位到高位逐位相加的过程。具体做法是：同时遍历两个链表，每次取当前节点的值相加，并加上进位 `carry`，得到当前位的结果 `sum % 10` 作为新节点的值，新的进位为 `sum // 10`。如果某个链表已经遍历结束，则其值视为 `0`。循环直到两个链表都遍历完且没有进位为止。

复杂度分析： 时间复杂度 $O(\max(m,n))$ ，空间复杂度 $O(\max(m,n))$ （新链表）。

Python代码：

```
# Definition for singly-linked list.
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

class Solution:
    def addTwoNumbers(self, l1, l2):
        dummy = ListNode(0)
        cur = dummy
        carry = 0

        while l1 or l2 or carry:
            v1 = l1.val if l1 else 0
            v2 = l2.val if l2 else 0

            s = v1 + v2 + carry
            carry = s // 10

            cur.next = ListNode(s % 10)
            cur = cur.next

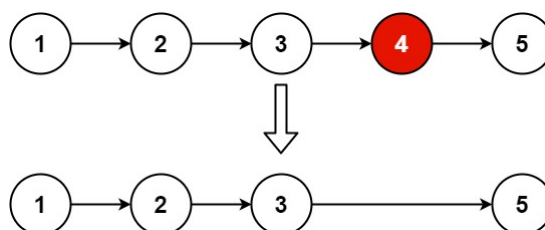
            if l1:
                l1 = l1.next
            if l2:
                l2 = l2.next

        return dummy.next
```

8. 删除链表的倒数第N个结点（中等）

给你一个链表，删除链表的倒数第 `n` 个结点，并且返回链表的头结点。

示例：



输入: `head = [1,2,3,4,5]`, `n = 2`

输出: `[1,2,3,5]`

解题思路：本题使用 **快慢指针（双指针）** 模板。思路是：先创建一个 **虚拟头节点 dummy**，方便删除头节点。定义两个指针 **fast** 和 **slow**，都指向 **dummy**。先让 **fast** 走 $n + 1$ 步，这样 **slow** 和 **fast** 之间就相隔 n 个节点。然后 **slow** 和 **fast** 同步向前移动，直到 **fast** 到达链表末尾。此时 **slow** 的下一个节点就是 **倒数第 n 个节点**，直接修改 **`slow.next = slow.next.next`** 即可删除该节点。

复杂度分析：时间复杂度 $O(n)$ ，空间复杂度 $O(1)$ 。

Python代码：

```
class Solution:
    def removeNthFromEnd(self, head: ListNode, n: int) -> ListNode:
        dummy = ListNode(0, head)
        slow = fast = dummy

        # 快指针先走 n+1 步
        for _ in range(n + 1):
            fast = fast.next

        # 快慢指针同时移动
        while fast:
            slow = slow.next
            fast = fast.next

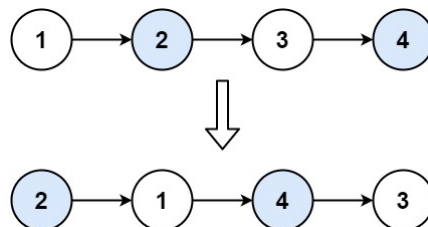
        # 删除倒数第 n 个节点
        slow.next = slow.next.next

        return dummy.next
```

9. 两两交换链表中的节点（中等）

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在**不修改节点内部的值**的情况下完成本题（即，只能进行节点交换）。

示例：



输入：head = [1,2,3,4]

输出：[2,1,4,3]

解题思路：本题要求 **交换链表中的相邻节点**，不能修改节点值，所以只能 **调整指针**。使用 **迭代 + 虚拟头节点** 的方法比较直观：创建一个 **虚拟头节点 dummy**，指向原链表头节点，方便操作头节点。用一个指针 **prev** 指向当前要交换的节点的前一个节点。每次取出两个节点 **first** 和 **second**：**`prev.next = second`**，**`first.next = second.next`**，**`second.next = first`**，更新 **prev** 指针，移动到下一个待交换的节点对。循环直到链表遍历完成。

复杂度分析：时间复杂度： $O(n)$ ，遍历每个节点一次空间复杂度： $O(1)$ ，只使用常数指针。

Python代码：

```

class Solution:
    def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0)
        dummy.next = head
        prev = dummy

        while prev.next and prev.next.next:
            first = prev.next
            second = first.next

            # 交换指针
            prev.next = second
            first.next = second.next
            second.next = first

            # 移动 prev
            prev = first

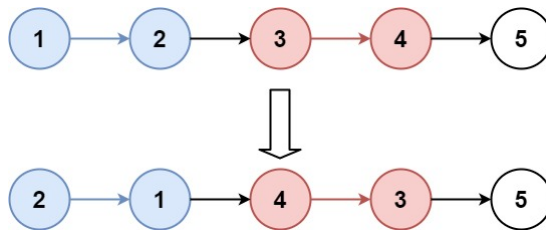
        return dummy.next

```

10. K个一组翻转链表（困难）

给你链表的头节点 `head`，每 `k` 个节点一组进行翻转，请你返回修改后的链表。`k` 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 `k` 的整数倍，那么请将最后剩余节点保持原有顺序。你不能只是单纯的改变节点内部的值，而是需要实际进行节点交换。

示例：



输入: `head = [1,2,3,4,5]`, `k = 2`
 输出: `[2,1,4,3,5]`

解题思路：本题是 **链表分组翻转** 的经典问题，可用 **迭代 + 虚拟头节点 + 局部反转** 的方法：创建 **虚拟头节点** `dummy` 指向 `head`，方便操作头节点。用 `prev_group` 指向每组翻转前的前一个节点。循环处理链表，每次检查 **剩余节点数是否 $\geq k$** ：如果不足 `k` 个，直接退出，保留剩余部分。找到本组的 **末尾节点** `kth`，然后翻转这 `k` 个节点：翻转方法可以用**头插法**：从 `group_prev.next` 开始，逐个移动节点到组头。翻转完成后，更新 `prev_group` 指针到本组末尾，准备下一组翻转。

复杂度分析：时间复杂度： $O(n)$ ，每个节点只处理一次。空间复杂度： $O(1)$ ，只使用指针。

Python代码：

```

class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        dummy = ListNode(0)
        dummy.next = head

```



```

prev_group = dummy

while True:
    # 找到本组的第 k 个节点
    kth = prev_group
    for _ in range(k):
        kth = kth.next
    if not kth:
        return dummy.next # 不足 k 个，直接返回

    group_next = kth.next

    # 反转本组
    prev, curr = kth.next, prev_group.next
    while curr != group_next:
        tmp = curr.next
        curr.next = prev
        prev = curr
        curr = tmp

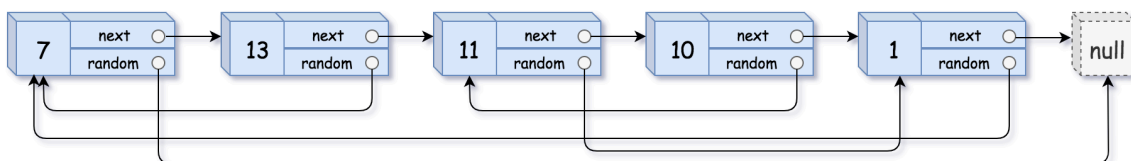
    # 连接上一组
    tmp = prev_group.next
    prev_group.next = kth
    prev_group = tmp

```

11. 随机链表的复制（中等）

给你一个长度为 n 的链表，每个节点包含一个额外增加的随机指针 `random`，该指针可以指向链表中的任何节点或空节点。构造这个链表的 **深拷贝**。深拷贝应该正好由 n 个 **全新** 节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。**复制链表中的指针都不应指向原链表中的节点**。例如，如果原链表中有 x 和 y 两个节点，其中 $x.random \rightarrow y$ 。那么在复制链表中对应的两个节点 x 和 y ，同样有 $x.random \rightarrow y$ 。返回复制链表的头节点。用一个由 n 个节点组成的链表来表示输入/输出中的链表。每个节点用一个 `[val, random_index]` 表示：`val`：一个表示 `Node.val` 的整数。`random_index`：随机指针指向的节点索引（范围从 0 到 $n-1$ ）；如果不指向任何节点，则为 `null`。你的代码 **只** 接受原链表的头节点 `head` 作为传入参数。

示例：



输入：head = [[7,null],[13,0],[11,4],[10,2],[1,0]]

输出：[[7,null],[13,0],[11,4],[10,2],[1,0]]

解题思路： 本题要求 **深拷贝带 random 指针的链表**，可以用 **三步法 + 原地操作** 实现，避免额外的哈希表空间（当然哈希表方法也可行， $O(n)$ 空间）：

1. 复制节点插入原链表:

遍历原链表, 将每个节点的拷贝节点插入到原节点的后面。

例如: `A -> B -> C` → `A -> A' -> B -> B' -> C -> C'`

2. 设置拷贝节点的 random 指针:

遍历新链表, 每个新节点 `node.next` 的 random 指针指向原节点的 random 的下一个节点 (即拷贝节点)。

```
if node.random:
    node.next.random = node.random.next
```

3. 拆分链表:

将原链表和复制链表拆开, 形成独立的新链表。

复杂度分析: 时间复杂度: $O(n)$, 遍历链表三次。空间复杂度: $O(1)$, 不使用额外哈希表。

Python代码:

```
# Definition for a Node.
class Node:
    def __init__(self, val: int = 0, next: 'Node' = None, random: 'Node' = None):
        self.val = val
        self.next = next
        self.random = random

class Solution:
    def copyRandomList(self, head: 'Node') -> 'Node':
        if not head:
            return None

        # 1. 复制节点插入原链表
        curr = head
        while curr:
            new_node = Node(curr.val, curr.next, None)
            curr.next = new_node
            curr = new_node.next

        # 2. 设置 random 指针
        curr = head
        while curr:
            if curr.random:
                curr.next.random = curr.random.next
            curr = curr.next.next

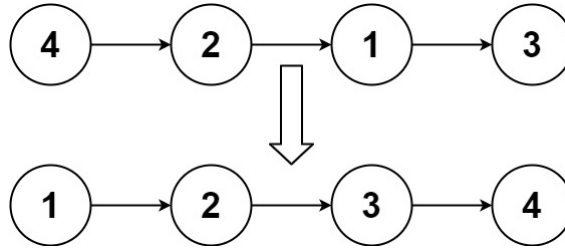
        # 3. 拆分链表
        curr = head
        copy_head = head.next
        while curr:
            copy = curr.next
            curr.next = copy.next
            copy.next = copy.next.next if copy.next else None
            curr = curr.next

        return copy_head
```

12. 排序链表（中等）

给你链表的头结点 `head`，请将其按 **升序** 排列并返回 **排序后的链表**。

示例：



输入：head = [4,2,1,3]

输出：[1,2,3,4]

解题思路： 本题要求在 $O(n \log n)$ 时间复杂度内对链表排序，通常使用 **归并排序**，因为链表随机访问不方便，不适合快速排序或堆排序。归并排序的核心步骤：

1. **找链表 midpoint：** 使用 **快慢指针** 找到链表的中点，将链表拆分成左右两半。
2. **递归排序左右链表：** 分别对左右两半链表递归进行排序。
3. **合并两个有序链表：** 使用类似 **合并两个有序链表** 的方法，将两个已排序链表合并成一个有序链表。

复杂度分析： 时间复杂度： $O(n \log n)$ ，链表拆分和合并。空间复杂度： $O(\log n)$ ，递归栈。

Python代码：

```
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if not head or not head.next:
            return head

        # 1. 找中点
        slow, fast = head, head.next
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next

        mid = slow.next
        slow.next = None # 切断链表

        # 2. 递归排序左右链表
        left = self.sortList(head)
        right = self.sortList(mid)

        # 3. 合并两个有序链表
        return self.merge(left, right)

    def merge(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
        dummy = ListNode(0)
        tail = dummy
```

```

while l1 and l2:
    if l1.val < l2.val:
        tail.next = l1
        l1 = l1.next
    else:
        tail.next = l2
        l2 = l2.next
    tail = tail.next

tail.next = l1 or l2
return dummy.next

```

13. 合并K个升序链表（困难）

给你一个链表数组，每个链表都已经按升序排列。请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例：

```

输入: lists = [[1,4,5],[1,3,4],[2,6]]
输出: [1,1,2,3,4,4,5,6]
解释: 链表数组如下:
[
  1->4->5,
  1->3->4,
  2->6
]
将它们合并到一个有序链表中得到。
1->1->2->3->4->4->5->6

```

解题思路：本题是 **多路归并** 的经典问题，主要有两种高效解法：

1. **优先队列/最小堆法（推荐）**：将每个链表的头节点加入一个 **最小堆**（按照节点值排序）。每次从堆里取最小节点，加入结果链表。如果取出的节点有下一个节点，则把下一个节点加入堆。时间复杂度 $O(N \log k)$ ，其中 N 是所有节点总数， k 是链表数量。
2. **两两合并法（归并）**：类似归并排序，每次两两合并链表，减少链表数量。时间复杂度 $O(N \log k)$ ，空间复杂度 $O(1)$ （迭代）或 $O(\log k)$ （递归）。

复杂度分析：时间复杂度： $O(N \log k)$ ， N 为所有节点总数， k 为链表数量。空间复杂度： $O(k)$ ，堆大小最多为 k 。

Python代码：

```

import heapq

class Solution:
    def mergeKLists(self, lists: list[Optional[ListNode]]) -> Optional[ListNode]:
        dummy = ListNode(0)
        tail = dummy
        heap = []

        # 初始化堆
        for idx, node in enumerate(lists):

```

```

        if node:
            heapq.heappush(heap, (node.val, idx, node))

    while heap:
        val, idx, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, idx, node.next))

    return dummy.next

```

14. LRU缓存 (中等)

请你设计并实现一个满足 [LRU \(最近最少使用\) 缓存](#) 约束的数据结构。实现 `LRUCache` 类：

- `LRUCache(int capacity)` 以 **正整数** 作为容量 `capacity` 初始化 LRU 缓存
- `int get(int key)` 如果关键字 `key` 存在于缓存中，则返回关键字的值，否则返回 `-1`。
- `void put(int key, int value)` 如果关键字 `key` 已经存在，则变更其数据值 `value`；如果不存在，则向缓存中插入该组 `key-value`。如果插入操作导致关键字数量超过 `capacity`，则应该 **逐出** 最久未使用的关键字。

函数 `get` 和 `put` 必须以 `O(1)` 的平均时间复杂度运行。

示例：

```

输入
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

输出
[null, null, null, 1, null, -1, null, -1, 3, 4]

```

解释

```

LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // 缓存是 {1=1}
lRUCache.put(2, 2); // 缓存是 {1=1, 2=2}
lRUCache.get(1);    // 返回 1
lRUCache.put(3, 3); // 该操作会使得关键字 2 作废，缓存是 {1=1, 3=3}
lRUCache.get(2);    // 返回 -1 (未找到)
lRUCache.put(4, 4); // 该操作会使得关键字 1 作废，缓存是 {4=4, 3=3}
lRUCache.get(1);    // 返回 -1 (未找到)
lRUCache.get(3);     // 返回 3
lRUCache.get(4);     // 返回 4

```

解题思路：实现 LRU 缓存需要同时支持 `O(1)` 时间复杂度的 `get` 和 `put` 操作。核心是使用 **哈希表 + 双向链表**：哈希表用于快速定位 `key` 对应的节点，双向链表维护节点的访问顺序，头部表示最近使用，尾部表示最久未使用。每次访问或插入节点时，将节点移动到链表头部，当容量超限时删除链表尾部节点。

复杂度分析：时间复杂度：`O(1)` 每次 `get` 或 `put`。空间复杂度：`O(capacity)`，用于存储哈希表和链表节点。

Python 代码：

```

class DLinkedNode:
    def __init__(self, key=0, value=0):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:

    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = dict() # key -> node
        self.size = 0
        # 虚拟头尾节点
        self.head = DLinkedNode()
        self.tail = DLinkedNode()
        self.head.next = self.tail
        self.tail.prev = self.head

    # 添加节点到头部
    def _add_node(self, node):
        node.prev = self.head
        node.next = self.head.next
        self.head.next.prev = node
        self.head.next = node

    # 删除节点
    def _remove_node(self, node):
        prev = node.prev
        nxt = node.next
        prev.next = nxt
        nxt.prev = prev

    # 移动节点到头部
    def _move_to_head(self, node):
        self._remove_node(node)
        self._add_node(node)

    # 弹出尾节点
    def _pop_tail(self):
        res = self.tail.prev
        self._remove_node(res)
        return res

    def get(self, key: int) -> int:
        node = self.cache.get(key, None)
        if not node:
            return -1
        self._move_to_head(node)
        return node.value

    def put(self, key: int, value: int) -> None:
        node = self.cache.get(key)
        if not node:
            newNode = DLinkedNode(key, value)

```

```

self.cache[key] = newNode
self._add_node(newNode)
self.size += 1
if self.size > self.capacity:
    tail = self._pop_tail()
    del self.cache[tail.key]
    self.size -= 1
else:
    node.value = value
    self._move_to_head(node)

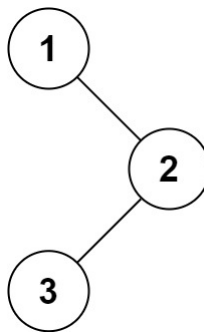
```

八. 二叉树

1. 二叉树的中序遍历（简单）

给定一个二叉树的根节点 `root`，返回 它的 **中序** 遍历。

示例：



输入: `root = [1,null,2,3]`

输出: `[1,3,2]`

解题思路：中序遍历的顺序是 **左 → 根 → 右**。可以使用递归或迭代方式实现。递归方法直观，将左子树遍历结果 + 根节点 + 右子树遍历结果拼接即可。迭代方法使用栈，每次将当前节点沿左子树压入栈，直到最左节点，然后弹出访问，转向右子树，直到遍历完整棵树。

复杂度分析：时间复杂度： $O(n)$ ，每个节点访问一次。空间复杂度： $O(n)$ ，递归栈或显式栈最多存储树的高度节点。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import List, Optional

class Solution:
    # 递归方法
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:

```

```

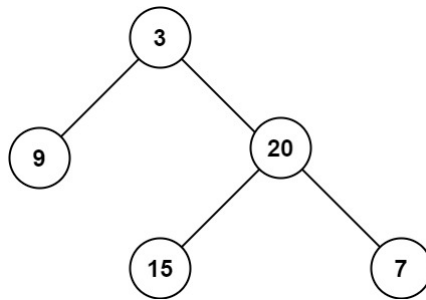
res = []
def dfs(node: Optional[TreeNode]):
    if not node:
        return
    dfs(node.left)
    res.append(node.val)
    dfs(node.right)
dfs(root)
return res

```

2. 二叉树的最大深度（简单）

给定一个二叉树 `root`，返回其最大深度。二叉树的 **最大深度** 是指从根节点到最远叶子节点的最长路径上的节点数。

示例：



输入: `root = [3,9,20,null,null,15,7]`

输出: 3

解题思路：二叉树最大深度可以通过 **递归** 或 **迭代（BFS）** 求解。递归方法：最大深度等于左子树和右子树深度的最大值 + 1。迭代方法：可以用 **队列进行层序遍历**，每遍历一层就增加深度计数，直到遍历完整棵树。

复杂度分析：时间复杂度： $O(n)$ ，每个节点访问一次。空间复杂度： $O(h)$ ，递归栈最大为树的高度 h ，BFS 队列也最多存储一层节点。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        left_depth = self.maxDepth(root.left)
        right_depth = self.maxDepth(root.right)

```

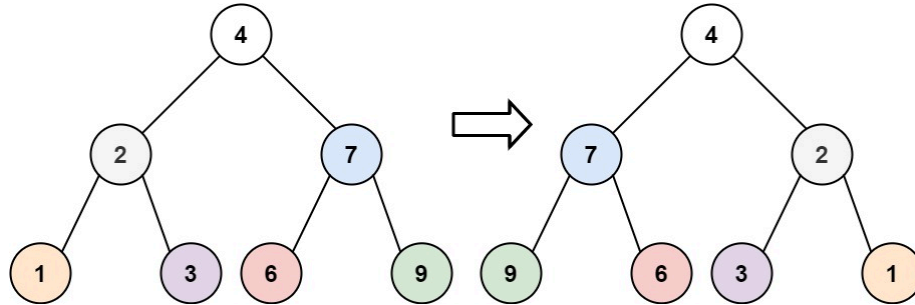


```
return max(left_depth, right_depth) + 1
```

3. 翻转二叉树（简单）

给你一棵二叉树的根节点 `root`，翻转这棵二叉树，并返回其根节点。

示例：



输入: `root = [4,2,7,1,3,6,9]`

输出: `[4,7,2,9,6,3,1]`

解题思路：翻转二叉树的本质是 **对每一个节点交换它的左子节点和右子节点**。可以使用 **递归** 实现：对于当前节点，先交换左右子树，然后递归地翻转左子树和右子树。当节点为空时直接返回。递归结束后，整棵树就完成了翻转。

复杂度分析：时间复杂度： $O(n)$ 每个节点都会被访问一次。空间复杂度： $O(h)$ 。递归调用栈的深度为树的高度 h ，最坏情况下为 $O(n)$ ，平均为 $O(\log n)$ 。

Python代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if not root:
            return None

        root.left, root.right = root.right, root.left

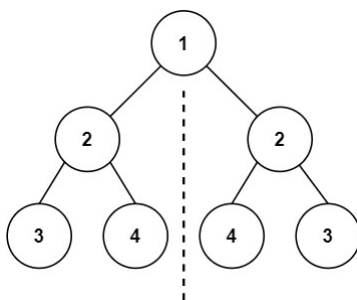
        self.invertTree(root.left)
        self.invertTree(root.right)

        return root
```

4. 对称二叉树（简单）

给你一个二叉树的根节点 `root`，检查它是否轴对称。

示例：



输入: `root = [1,2,2,3,4,4,3]`

输出: `true`

解题思路：判断一棵二叉树是否轴对称，本质是判断 **左子树和右子树是否互为镜像**。两个子树互为镜像需要满足三个条件：

1. 两个节点的值相同
2. 左子树的左节点等于右子树的右节点
3. 左子树的右节点等于右子树的左节点

因此可以写一个递归函数 `isMirror(left, right)` 来判断两个节点是否对称：

- 如果两个节点都为空，说明对称
- 如果一个为空一个不为空，不对称
- 如果值不同，不对称
- 否则递归比较 `left.left` 和 `right.right` 以及 `left.right` 和 `right.left`

复杂度分析：时间复杂度： $O(n)$ ，每个节点最多被访问一次。空间复杂度： $O(h)$ ，递归调用栈深度为树的高度 h ，最坏情况为 $O(n)$ 。

Python代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def isMirror(left, right):
            if not left and not right:
                return True
            if not left or not right:
                return False
            if left.val != right.val:
                return False
```

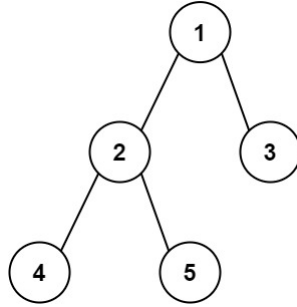
```
        return isMirror(left.left, right.right) and isMirror(left.right,
right.left)

    return isMirror(root.left, root.right)
```

5. 二叉树的直径（简单）

给你一棵二叉树的根节点，返回该树的 **直径**。二叉树的 **直径** 是指树中任意两个节点之间最长路径的 **长度**。这条路径可能经过也可能不经过根节点 `root`。两节点之间路径的 **长度** 由它们之间边数表示。

示例：



输入：root = [1,2,3,4,5]

输出：3

解释：3，取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

解题思路：二叉树的直径表示树中任意两个节点之间最长路径的边数，这条路径可能经过根节点，也可能不经过。对于每个节点，如果知道其左子树的最大深度和右子树的最大深度，那么经过该节点的最长路径就是左子树深度加右子树深度。因此可以使用深度优先搜索递归计算每个节点的深度，在计算深度的同时更新当前可能的最大路径长度。递归函数返回当前节点的最大深度 `max(left_depth, right_depth) + 1`，并用一个变量记录遍历过程中出现的最大直径，最终得到整棵树的直径。

复杂度分析：时间复杂度为 $O(n)$ ，因为每个节点只会被遍历一次，并在遍历过程中同时计算深度和更新直径。空间复杂度为 $O(h)$ ，其中 h 为二叉树的高度，主要来自递归调用栈的空间开销；最坏情况下树退化为链表时为 $O(n)$ ，平均情况下为 $O(\log n)$ 。

Python代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def diameterofBinaryTree(self, root: Optional[TreeNode]) -> int:
        self.diameter = 0

        def depth(node):
            if not node:
                return 0

            left = depth(node.left)
            right = depth(node.right)
            self.diameter = max(self.diameter, left + right + 1)
            return max(left, right) + 1

        depth(root)
```

```

        left = depth(node.left)
        right = depth(node.right)

        self.diameter = max(self.diameter, left + right)

    return max(left, right) + 1

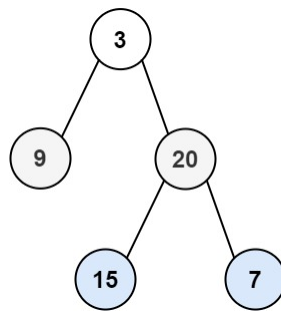
depth(root)
return self.diameter

```

6. 二叉树的层序遍历（中等）

给你二叉树的根节点 `root`，返回其节点值的 **层序遍历**。（即逐层地，从左到右访问所有节点）。

示例：



输入: `root = [3,9,20,null,null,15,7]`
 输出: `[[3],[9,20],[15,7]]`

解题思路：二叉树的层序遍历就是按照从上到下、从左到右的顺序逐层访问节点，最适合使用广度优先搜索（BFS）实现。具体做法是使用一个队列，先将根节点加入队列，然后每次遍历当前队列中的所有节点（即当前层），依次取出节点并记录其值，同时将它们的左子节点和右子节点加入队列。这样一层一层遍历，每一层的节点值组成一个列表，最终得到所有层的结果列表。

复杂度分析：时间复杂度为 $O(n)$ ，因为每个节点都会被访问一次。空间复杂度为 $O(n)$ ，最坏情况下队列中需要存储一整层的节点，例如在完全二叉树的最后一层。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional, List
from collections import deque

class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []

        res = []
        queue = deque([root])

```

```

while queue:
    level = []
    for _ in range(len(queue)):
        node = queue.popleft()
        level.append(node.val)

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

    res.append(level)

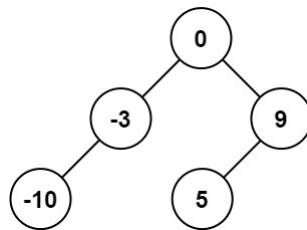
return res

```

7. 将有序数组转换为二叉搜索树（简单）

给你一个整数数组 `nums`，其中元素已经按 **升序** 排列，请你将其转换为一棵 **平衡** 二叉搜索树。

示例：



输入: `nums = [-10,-3,0,5,9]`

输出: `[0,-3,9,-10,null,5]`

解释: `[0,-10,5,null,-3,null,9]` 也将被视为正确答案：

解题思路：由于给定数组是升序排列，并且要求构造一棵平衡二叉搜索树，因此可以利用二叉搜索树的性质：左子树的值小于根节点，右子树的值大于根节点。为了保证树的平衡，可以选择数组的中间元素作为根节点，这样左右两部分长度大致相等。然后递归地对左半部分构建左子树，对右半部分构建右子树，直到子数组为空时返回 `None`。这样最终构建出的树就是一棵高度平衡的二叉搜索树。

复杂度分析：时间复杂度为 $O(n)$ ，因为数组中的每个元素都会被访问并创建一个树节点一次。空间复杂度为 $O(\log n)$ ，主要来自递归调用栈的深度，在平衡情况下递归深度约为 $\log n$ 。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional, List

class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
        def build(left, right):

```

```

        if left > right:
            return None

        mid = (left + right) // 2
        node = TreeNode(nums[mid])

        node.left = build(left, mid - 1)
        node.right = build(mid + 1, right)

        return node

    return build(0, len(nums) - 1)

```

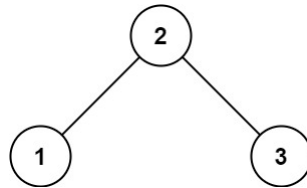
8. 验证二叉搜索树（中等）

给你一个二叉树的根节点 `root`，判断其是否是一个有效的二叉搜索树。

有效 二叉搜索树定义如下：

- 节点的左子树只包含 **严格小于** 当前节点的数。
- 节点的右子树只包含 **严格大于** 当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

示例：



输入: `root = [2,1,3]`

输出: `true`

解题思路：二叉搜索树具有一个重要性质：对其进行中序遍历时，得到的节点值序列一定是严格递增的。因此可以利用这一性质，通过中序遍历整棵树，并记录上一个访问的节点值。如果当前节点的值小于或等于上一个节点值，则说明不满足二叉搜索树的性质，直接返回 `False`；否则继续遍历。当遍历完整棵树都满足严格递增时，说明该树是一个有效的二叉搜索树。

复杂度分析：时间复杂度为 $O(n)$ ，因为需要对整棵树进行一次中序遍历，每个节点访问一次。空间复杂度为 $O(h)$ ，其中 h 为二叉树高度，主要来自递归调用栈的空间开销；最坏情况下树退化为链表时为 $O(n)$ 。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

```

```

class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        self.prev = float('-inf')

        def inorder(node):
            if not node:
                return True

            if not inorder(node.left):
                return False

            if node.val <= self.prev:
                return False
            self.prev = node.val

            return inorder(node.right)

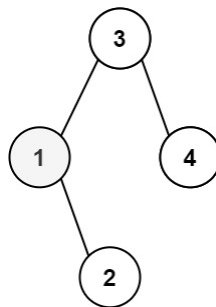
        return inorder(root)

```

9. 二叉搜索树中第K小的元素（中等）

给定一个二叉搜索树的根节点 `root`，和一个整数 `k`，请你设计一个算法查找其中第 `k` 小的元素（`k` 从 1 开始计数）。

示例：



输入: `root = [3,1,4,null,2]`, `k = 1`
 输出: 1

解题思路：二叉搜索树（BST）的中序遍历结果是从小到大的有序序列。因此可以通过中序遍历树，在遍历过程中计数，当计数等于 `k` 时，当前节点就是第 `k` 小的元素。可以使用递归或迭代的方式实现中序遍历，利用计数器判断是否到达第 `k` 个节点即可。

复杂度分析：时间复杂度： $O(H + k)$ ，其中 H 是树的高度。最坏情况下 $H = n$ （树退化为链表），最好情况为 $O(\log n + k)$ 。空间复杂度： $O(H)$ ，递归调用栈或迭代栈的空间开销。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

```

```

class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        self.count = 0
        self.result = None

        def inorder(node):
            if not node or self.result is not None:
                return

            inorder(node.left)

            self.count += 1
            if self.count == k:
                self.result = node.val
                return

            inorder(node.right)

        inorder(root)
        return self.result

```

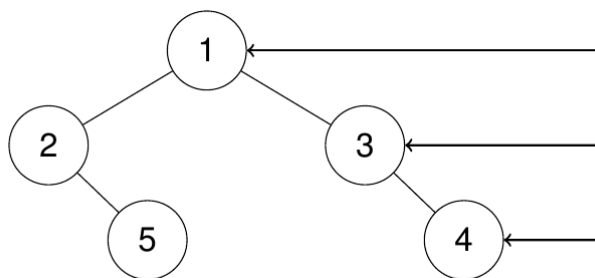
10. 二叉树的右视图（中等）

给定一个二叉树的 **根节点** `root`，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例：

输入: `root = [1,2,3,null,5,null,4]`
 输出: `[1,3,4]`

解释：



解题思路：二叉树的右视图是每一层最右边的节点值。可以使用 **层序遍历（BFS）**，在遍历每一层时记录该层的最后一个节点值，即为右视图中该层的可见节点。也可以用 **深度优先搜索（DFS）** 的方式，优先访问右子节点，每一层第一次访问的节点就是该层的右侧节点。

复杂度分析：时间复杂度： $O(n)$ ，每个节点访问一次。空间复杂度： $O(w)$ ，BFS 队列存储每一层节点， w 为树的最大宽度。递归 DFS 也为 $O(h)$ ， h 为树高。

Python代码：

```

# Definition for a binary tree node.

```



```

# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional, List
from collections import deque

class Solution:
    def rightSideView(self, root: Optional[TreeNode]) -> List[int]:
        if not root:
            return []

        result = []
        queue = deque([root])

        while queue:
            level_size = len(queue)
            for i in range(level_size):
                node = queue.popleft()
                # 每层最后一个节点加入结果
                if i == level_size - 1:
                    result.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)

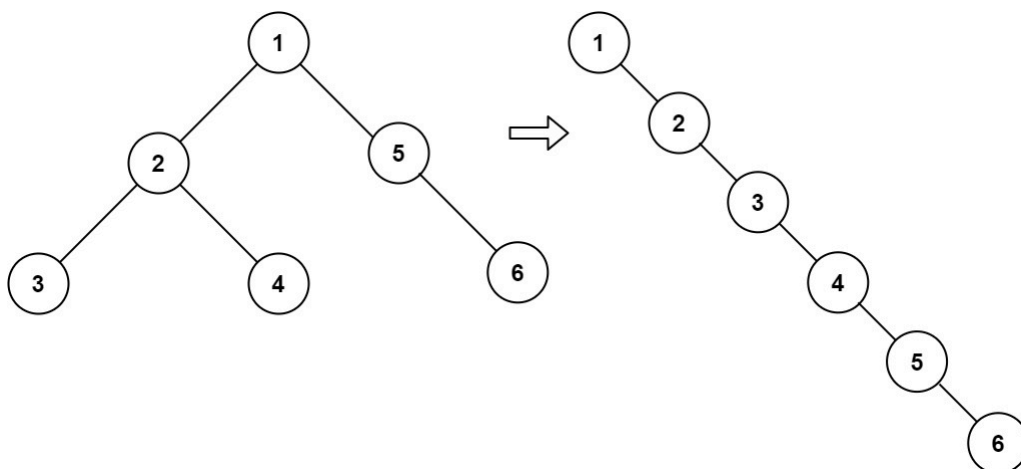
        return result

```

11. 二叉树展开为链表 (中等)

给你二叉树的根结点 `root`，请你将它展开为一个单链表：展开后的单链表应该同样使用 `TreeNode`，其中 `right` 子指针指向链表中下一个结点，而左子指针始终为 `null`。展开后的单链表应该与二叉树先序遍历顺序相同。

示例：



输入: `root = [1,2,5,3,4,null,6]`

输出: `[1,null,2,null,3,null,4,null,5,null,6]`

解题思路：本题要求将二叉树按照 **先序遍历** 展开为单链表，并且只能使用 `right` 指针指向下一个节点，`left` 指针置为 `None`。可以使用 **递归** 的方式：对每个节点，先递归展开左子树和右子树，然后将左子树接到当前节点的右子树位置，并把原右子树接到左子树的最右节点后面。也可以使用 **迭代 + 栈** 的方式模拟先序遍历，然后重新连接节点。

复杂度分析：时间复杂度： $O(n)$ ，每个节点访问一次。空间复杂度： $O(h)$ ，递归调用栈深度为树高 h ，最坏 $O(n)$ 。

Python代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        if not root:
            return

        # 递归展开左、右子树
        self.flatten(root.left)
        self.flatten(root.right)

        # 保存原右子树
        tmp_right = root.right

        # 将左子树移动到右子树位置
        root.right = root.left
        root.left = None

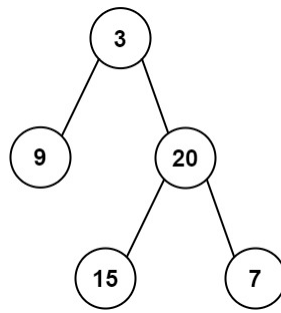
        # 找到新右子树的末端节点
        current = root
        while current.right:
            current = current.right

        # 接回原右子树
        current.right = tmp_right
```

12. 从前序与中序遍历序列构造二叉树（中等）

给定两个整数数组 `preorder` 和 `inorder`，其中 `preorder` 是二叉树的**先序遍历**，`inorder` 是同一棵树**的中序遍历**，请构造二叉树并返回其根节点。

示例：



输入: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
输出: [3,9,20,null,null,15,7]

解题思路: 利用二叉树的 **前序遍历** 和 **中序遍历** 的性质: 前序遍历的第一个元素是树的根节点; 在中序遍历中, 根节点左边的元素属于左子树, 右边的元素属于右子树。可以递归地对左右子树进行构造。为了快速查找中序中根节点的位置, 可以用哈希表保存中序值到索引的映射。

复杂度分析: 时间复杂度: $O(n)$, 每个节点只访问一次。空间复杂度: $O(n)$, 用于哈希表和递归调用栈, 最坏为树高 $O(n)$ 。

Python代码:

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import List, Optional

class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        if not preorder or not inorder:
            return None

        # 建立中序值到索引的映射, 快速定位根节点
        idx_map = {val: idx for idx, val in enumerate(inorder)}

        def helper(pre_left, pre_right, in_left, in_right):
            if pre_left > pre_right:
                return None

            # 前序遍历的第一个节点是根节点
            root_val = preorder[pre_left]
            root = TreeNode(root_val)

            # 根节点在中序遍历中的索引
            in_root_idx = idx_map[root_val]

            # 左子树节点个数
            left_size = in_root_idx - in_left

            # 构造左子树
            root.left = helper(pre_left + 1, pre_left + left_size, in_left,
                               in_root_idx - 1)
```

```

        # 构造右子树
        root.right = helper(pre_left + left_size + 1, pre_right, in_root_idx
+ 1, in_right)

    return root

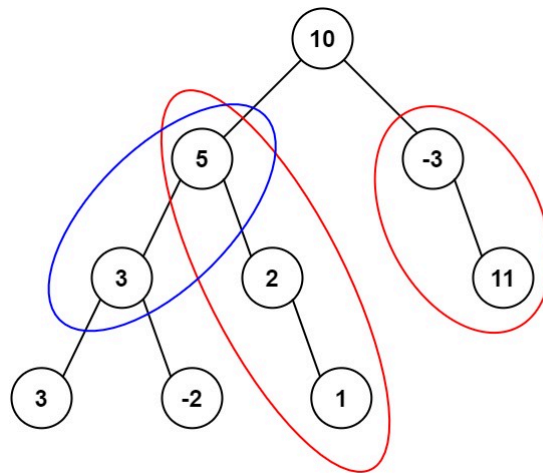
return helper(0, len(preorder) - 1, 0, len(inorder) - 1)

```

13. 路径总和III (中等)

给定一个二叉树的根节点 `root`，和一个整数 `targetSum`，求该二叉树里节点值之和等于 `targetSum` 的 **路径** 的数目。**路径** 不需要从根节点开始，也不需要叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

示例：



输入: `root = [10,5,-3,3,2,null,11,3,-2,null,1]`, `targetSum = 8`
 输出: 3
 解释: 和等于 8 的路径有 3 条，如图所示。

解题思路：这个题需要统计二叉树中向下路径和为 `targetSum` 的数量。可以使用 **前缀和 + 哈希表** 的方法：遍历树时记录从根到当前节点的路径和 `curr_sum`，用哈希表 `prefix` 统计前缀和出现的次数。对于当前节点，路径和等于 `targetSum` 的数量等于 `prefix[curr_sum - targetSum]`。递归遍历左子树和右子树，并在递归返回时恢复哈希表，保证路径正确性。这样可以在 $O(n)$ 时间内完成统计。

复杂度分析：时间复杂度： $O(n)$ ，每个节点访问一次。空间复杂度： $O(n)$ ，哈希表存储前缀和，递归调用栈最坏为 $O(n)$ 。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:

```

```
def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
    from collections import defaultdict

    prefix = defaultdict(int)
    prefix[0] = 1 # 路径和正好等于 targetSum 的情况

    def dfs(node, curr_sum):
        if not node:
            return 0

        curr_sum += node.val
        # 当前节点路径和等于 targetSum 的数量
        count = prefix[curr_sum - targetSum]

        # 更新前缀和
        prefix[curr_sum] += 1

        # 遍历左右子树
        count += dfs(node.left, curr_sum)
        count += dfs(node.right, curr_sum)

        # 回溯，恢复前缀和
        prefix[curr_sum] -= 1

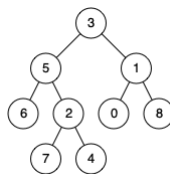
    return count

return dfs(root, 0)
```

14. 二叉树的最近公共祖先（中等）

给定一个二叉树，找到该树中两个指定节点的最近公共祖先。最近公共祖先的定义为：“对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。”

示例：



输入：root = [3,5,1,6,2,0,8,null,null,7,4]，p = 5，q = 1

输出：3

解释：节点 5 和节点 1 的最近公共祖先是节点 3。

解题思路：可以使用 **递归后序遍历** 的方式。对于当前节点 **root**，递归查找左右子树中是否包含 **p** 或 **q**。有三种情况：左右子树都找到了 **p** 和 **q**，则当前节点就是最近公共祖先。只在左子树找到，则祖先在左子树；只在右子树找到，则祖先在右子树。当前节点就是 **p** 或 **q**，则返回当前节点给父节点判断。通过递归后序遍历，可以在找到节点的同时逐层返回最近公共祖先。

复杂度分析：时间复杂度：**O(n)**，每个节点访问一次。空间复杂度：**O(h)**，递归调用栈的深度为树高 **h**，最坏情况为 **O(n)**。

Python代码：

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        if not root or root == p or root == q:
            return root

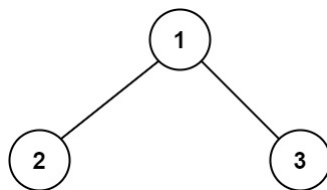
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)

        if left and right:
            return root
        return left if left else right
```

15. 二叉树中的最大路径和（困难）

二叉树中的 **路径** 被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 **至多出现一次**。该路径 **至少包含一个** 节点，且不一定经过根节点。路径和是路径中各节点值的总和。给你一个二叉树的根节点 `root`，返回其 **最大路径和**。

示例：



输入：root = [1,2,3]

输出：6

解释：最优路径是 2 -> 1 -> 3，路径和为 2 + 1 + 3 = 6

解题思路：这个题目要求找到二叉树中任意路径的最大和，路径可以从任意节点开始，也可以结束于任意节点，但同一个节点不能重复使用。可以使用 **后序遍历**（左右子树先计算）+ 递归的方式：对于当前节点 `root`，计算左右子树能贡献的最大路径和 `left_max`、`right_max`（如果为负数就取 0，表示不选择该子树）。当前节点的最大路径和为 `root.val + left_max + right_max`，更新全局变量 `max_sum`。返回给父节点的最大贡献值为 `root.val + max(left_max, right_max)`，表示只能选择左右子树之一接到父节点。

复杂度分析：时间复杂度： $O(n)$ ，每个节点访问一次。空间复杂度： $O(h)$ ，递归栈空间为树的高度，最坏情况为 $O(n)$ 。

Python代码：

```

# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
from typing import Optional

class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        self.max_sum = float('-inf')

        def helper(node: Optional[TreeNode]) -> int:
            if not node:
                return 0
            # 左右子树最大贡献值，如果为负就舍弃
            left_max = max(helper(node.left), 0)
            right_max = max(helper(node.right), 0)

            # 更新全局最大路径和
            self.max_sum = max(self.max_sum, node.val + left_max + right_max)

            # 返回给父节点的最大贡献值
            return node.val + max(left_max, right_max)

        helper(root)
        return self.max_sum

```

九. 图论

1. 岛屿数量（中等）

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。此外，你可以假设该网格的四条边均被水包围。

示例：

```

输入: grid = [
  ['1','1','1','1','0'],
  ['1','1','0','1','0'],
  ['1','1','0','0','0'],
  ['0','0','0','0','0']
]
输出: 1

```

解题思路：这个题目可以抽象为图中的连通分量问题，每个岛屿就是一个连通分量。可以使用 **深度优先搜索（DFS）** 或 **广度优先搜索（BFS）** 来遍历岛屿。遍历到一个陆地 '1' 时，将它标记为 '0'（或访问过），然后递归/入队检查其上下左右相邻的陆地，直到整个岛屿被遍历完。每发现一次新的陆地，就计数加一。

复杂度分析：时间复杂度： $O(m*n)$ ，每个格子访问一次。空间复杂度： $O(m*n)$ ，最坏情况下 DFS 递归栈或 BFS 队列可能需要存储整个网格。

Python代码：

```
from typing import List

class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid:
            return 0

        m, n = len(grid), len(grid[0])
        count = 0

        def dfs(i: int, j: int):
            if i < 0 or i >= m or j < 0 or j >= n or grid[i][j] == '0':
                return
            grid[i][j] = '0' # 标记已访问
            # 上下左右
            dfs(i-1, j)
            dfs(i+1, j)
            dfs(i, j-1)
            dfs(i, j+1)

        for i in range(m):
            for j in range(n):
                if grid[i][j] == '1':
                    count += 1
                    dfs(i, j)

        return count
```

2. 腐烂的橘子（中等）

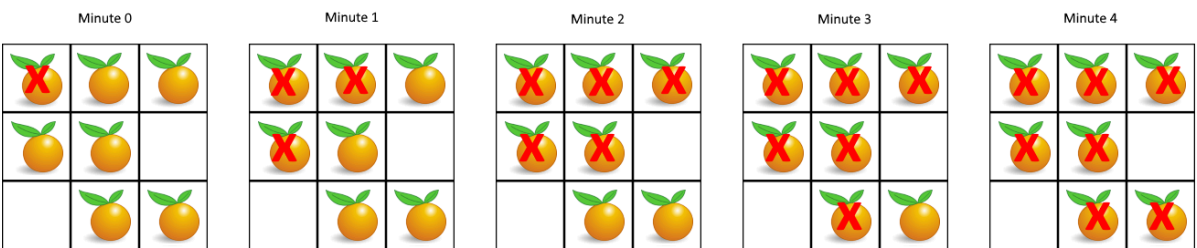
在给定的 `m x n` 网格 `grid` 中，每个单元格可以有以下三个值之一：

- 值 `0` 代表空单元格；
- 值 `1` 代表新鲜橘子；
- 值 `2` 代表腐烂的橘子。

每分钟，腐烂的橘子 **周围 4 个方向上相邻** 的新鲜橘子都会腐烂。

返回 *直到单元格中没有新鲜橘子为止所必须经过的最小分钟数*。如果不可能，返回 `-1`。

示例：



输入: grid = [[2,1,1],[1,1,0],[0,1,1]]
输出: 4

解题思路: 这是典型的 **多源 BFS** 问题。将所有腐烂橘子 (值为 2) 的位置加入队列, 表示这些橘子在第 0 分钟开始腐烂。每分钟从队列中取出所有当前腐烂的橘子, 然后将它们上下左右的新鲜橘子腐烂 (值变为 2) 并加入队列。每轮 BFS 代表一分钟。遍历结束后, 如果仍有新鲜橘子存在, 则返回 -1, 否则返回所需分钟数。

复杂度分析: 时间复杂度: $O(m*n)$, 每个格子最多访问一次。空间复杂度: $O(m*n)$, 队列在最坏情况下存储整个网格的橘子。

Python代码:

```
from typing import List
from collections import deque

class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        if not grid:
            return -1

        m, n = len(grid), len(grid[0])
        queue = deque()
        fresh_count = 0

        # 初始化队列和新鲜橘子计数
        for i in range(m):
            for j in range(n):
                if grid[i][j] == 2:
                    queue.append((i, j))
                elif grid[i][j] == 1:
                    fresh_count += 1

        if fresh_count == 0:
            return 0

        minutes = 0
        directions = [(-1,0),(1,0),(0,-1),(0,1)]

        while queue:
            size = len(queue)
            changed = False # 本轮是否有新鲜橘子腐烂
            for _ in range(size):
                x, y = queue.popleft()
                for dx, dy in directions:
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < m and 0 <= ny < n and grid[nx][ny] == 1:
                        grid[nx][ny] = 2
                        fresh_count -= 1
                        queue.append((nx, ny))
                        changed = True
            if changed:
                minutes += 1
```

```
return minutes if fresh_count == 0 else -1
```

3. 课程表（中等）

你这个学期必须选修 `numCourses` 门课程，记为 `0` 到 `numCourses - 1`。

在选修某些课程之前需要一些先修课程。先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [ai, bi]`，表示如果要学习课程 `ai` 则 **必须** 先学习课程 `bi`。

- 例如，先修课程对 `[0, 1]` 表示：想要学习课程 `0`，你需要先完成课程 `1`。

请你判断是否可能完成所有课程的学习？如果可以，返回 `true`；否则，返回 `false`。

示例：

输入：`numCourses = 2, prerequisites = [[1,0]]`

输出：`true`

解释：总共有 2 门课程。学习课程 1 之前，你需要完成课程 0。这是可能的。

解题思路：这个问题可以抽象为一个**有向图是否存在环**的问题。每门课程是一个节点，先修关系 `[a, b]` 表示一条从 `b → a` 的有向边。如果图中存在环，就说明存在课程互相依赖，无法完成所有课程。可以使用**拓扑排序（BFS + 入度统计）**来解决：先统计每个节点的入度，并把所有入度为 0 的节点加入队列，然后不断从队列中取出节点并减少其邻接节点的入度。如果某个节点入度变为 0，则加入队列。最后如果能够访问完所有课程，则说明没有环，可以完成所有课程，否则存在环。

复杂度分析：时间复杂度为 $O(V + E)$ ，其中 V 为课程数量， E 为先修关系数量，每条边和每个节点都只处理一次。空间复杂度为 $O(V + E)$ ，用于存储邻接表、入度数组以及队列。

Python代码：

```
from typing import List
from collections import deque, defaultdict

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        graph = defaultdict(list)
        indegree = [0] * numCourses

        # 构建图和入度表
        for a, b in prerequisites:
            graph[b].append(a)
            indegree[a] += 1

        queue = deque()

        # 入度为0的课程先入队
        for i in range(numCourses):
            if indegree[i] == 0:
                queue.append(i)

        count = 0

        while queue:
            course = queue.popleft()
```

```

count += 1

for neighbor in graph[course]:
    indegree[neighbor] -= 1
    if indegree[neighbor] == 0:
        queue.append(neighbor)

return count == numCourses

```

4. 实现Trie (前缀树) (中等)

Trie (发音类似 "try") 或者说 **前缀树** 是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补全和拼写检查。

请你实现 Trie 类：

- `Trie()` 初始化前缀树对象。
- `void insert(String word)` 向前缀树中插入字符串 `word`。
- `boolean search(String word)` 如果字符串 `word` 在前缀树中，返回 `true`（即，在检索之前已经插入）；否则，返回 `false`。
- `boolean startswith(String prefix)` 如果之前已经插入的字符串 `word` 的前缀之一为 `prefix`，返回 `true`；否则，返回 `false`。

示例：

```

输入
["Trie", "insert", "search", "search", "startswith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]

输出
[null, null, true, false, true, null, true]

```

```

解释
Trie trie = new Trie();
trie.insert("apple");
trie.search("apple"); // 返回 True
trie.search("app");   // 返回 False
trie.startswith("app"); // 返回 True
trie.insert("app");
trie.search("app");    // 返回 True

```

解题思路：Trie (前缀树) 是一种专门用于存储字符串的数据结构，每个节点表示一个字符。每个节点包含一个字典（或数组）用于存储子节点，以及一个标记表示该节点是否是某个单词的结尾。`insert` 操作逐字符遍历单词，如果当前字符不存在子节点就创建新节点；`search` 操作逐字符查找，如果中途不存在对应节点则返回 `False`，最后判断是否是单词结尾；`startswith` 与 `search` 类似，但只需要判断前缀是否存在，不要求必须是单词结尾。

复杂度分析：时间复杂度：`insert`、`search` 和 `startswith` 都为 $O(L)$ ，其中 L 为字符串长度。空间复杂度为 $O(N \times L)$ ，其中 N 为插入的单词数量， L 为平均单词长度，用于存储 Trie 节点。

Python代码：

```

class Trie:

```

```

def __init__(self):
    self.children = {}
    self.is_end = False

def insert(self, word: str) -> None:
    node = self
    for ch in word:
        if ch not in node.children:
            node.children[ch] = Trie()
        node = node.children[ch]
    node.is_end = True

def search(self, word: str) -> bool:
    node = self
    for ch in word:
        if ch not in node.children:
            return False
        node = node.children[ch]
    return node.is_end

def startswith(self, prefix: str) -> bool:
    node = self
    for ch in prefix:
        if ch not in node.children:
            return False
        node = node.children[ch]
    return True

```

十. 回溯

1. 全排列（中等）

给定一个不含重复数字的数组 `nums`，返回其 *所有可能的全排列*。你可以 **按任意顺序** 返回答案。

示例：

输入：nums = [1,2,3]

输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

解题思路：全排列问题可以使用 **回溯算法（Backtracking）** 解决。回溯的核心思想是逐步构建排列，每一步选择一个还没有使用过的数字加入当前路径，当路径长度等于数组长度时，就得到一个完整排列并加入结果。之后回退一步（撤销选择），继续尝试其他数字。为了避免重复使用元素，可以使用一个 `used` 数组记录当前元素是否已经被选过。

复杂度分析：时间复杂度为 $O(n \times n!)$ ，因为共有 $n!$ 种排列，每个排列需要 $O(n)$ 时间复制到结果列表。空间复杂度为 $O(n)$ ，主要来自递归调用栈和 `used` 数组（不包括结果存储）。

Python代码：

```
from typing import List
```

```

class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        res = []
        path = []
        used = [False] * len(nums)

        def backtrack():
            if len(path) == len(nums):
                res.append(path[:])
                return

            for i in range(len(nums)):
                if used[i]:
                    continue

                path.append(nums[i])
                used[i] = True

                backtrack()

                path.pop()
                used[i] = False

        backtrack()
        return res

```

2. 子集 (中等)

给你一个整数数组 `nums`，数组中的元素 **互不相同**。返回该数组所有可能的子集（幂集）。解集 **不能** 包含重复的子集。你可以按 **任意顺序** 返回解集。

示例：

```

输入: nums = [1,2,3]
输出: [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]

```

解题思路：子集问题可以使用 **回溯算法 (Backtracking)** 解决。对于数组中的每个元素，都有两种选择：加入当前子集或不加入。可以通过递归逐步构建子集，每到一个位置时先把当前路径加入结果，然后继续向后选择元素。为了避免重复，只从当前索引 `start` 之后的元素继续搜索，这样每个元素只会在其后的位置被选择，最终可以得到所有可能的子集。

复杂度分析：时间复杂度为 $O(n \times 2^n)$ ，因为共有 2^n 个子集，每个子集在加入结果时需要 $O(n)$ 时间复制路径。空间复杂度为 $O(n)$ ，主要来自递归调用栈和当前路径 `path`（不包括结果存储）。

Python代码：

```

from typing import List

class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        res = []
        path = []

```

```
def backtrack(start):
    res.append(path[:])

    for i in range(start, len(nums)):
        path.append(nums[i])
        backtrack(i + 1)
        path.pop()

backtrack(0)
return res
```

3. 电话号码的字母组合（中等）

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。答案可以按 **任意顺序** 返回。给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。



示例：

```
输入: digits = "23"
输出: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

解题思路： 这个问题可以使用 **回溯算法 (Backtracking)**。每个数字都对应多个字母，例如 2 -> abc、3 -> def。可以从第一个数字开始，依次选择对应的字母加入当前路径，然后递归处理下一个数字。当路径长度等于输入字符串长度时，说明形成了一个完整的字母组合，将其加入结果。不断回溯尝试所有可能的组合即可。

复杂度分析： 时间复杂度为 $O(4^n)$ ，其中 n 是 `digits` 的长度，每个数字最多对应 4 个字母（如 7 和 9）。空间复杂度为 $O(n)$ ，主要来自递归调用栈和当前路径（不包括结果存储）。

Python代码：

```
from typing import List

class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        if not digits:
            return []
```

```

phone = {
    "2": "abc", "3": "def", "4": "ghi",
    "5": "jkl", "6": "mno", "7": "pqrs",
    "8": "tuv", "9": "wxyz"
}

res = []
path = []

def backtrack(index):
    if index == len(digits):
        res.append("".join(path))
        return

    letters = phone[digits[index]]
    for ch in letters:
        path.append(ch)
        backtrack(index + 1)
        path.pop()

backtrack(0)
return res

```

4. 组合总和（中等）

给你一个 **无重复元素** 的整数数组 `candidates` 和一个目标整数 `target`，找出 `candidates` 中可以使数字和为目标数 `target` 的所有 **不同组合**，并以列表形式返回。你可以按 **任意顺序** 返回这些组合。

`candidates` 中的 **同一个** 数字可以 **无限制重复被选取**。如果至少一个数字的被选数量不同，则两种组合是不同的。对于给定的输入，保证和为 `target` 的不同组合数少于 150 个。

示例：

```

输入: candidates = [2,3,6,7], target = 7
输出: [[2,2,3],[7]]
解释:
2 和 3 可以形成一组候选，2 + 2 + 3 = 7 。注意 2 可以使用多次。
7 也是一个候选， 7 = 7 。
仅有这两种组合。

```

解题思路： 本题可以使用 **回溯算法（Backtracking）**。在每一步从 `candidates` 中选择一个数字加入当前组合，并更新当前和。当当前和等于 `target` 时，将该组合加入结果；如果当前和超过 `target`，则停止继续搜索。因为题目允许同一个数字被重复选择，所以递归时仍然从当前索引开始搜索（而不是 `i+1`），这样可以继续选择当前数字。同时通过 `start` 索引保证组合不会重复，例如 `[2,3]` 和 `[3,2]` 只会生成一种。

复杂度分析： 时间复杂度为 $O(N^{(target/min)})$ 的回溯复杂度，其中 `min` 为最小候选数，实际复杂度取决于搜索树规模。空间复杂度为 $O(target)$ ，主要来自递归调用栈和当前路径 `path`（不包括结果存储）。

Python代码：

```

from typing import List

```

```

class Solution:
    def combinationSum(self, candidates: List[int], target: int) ->
List[List[int]]:
    res = []
    path = []

    def backtrack(start, total):
        if total == target:
            res.append(path[:])
            return
        if total > target:
            return

        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, total + candidates[i]) # 可以重复使用当前数字
            path.pop()

    backtrack(0, 0)
    return res

```

5. 括号生成 (中等)

数字 n 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且 **有效的** 括号组合。

示例：

```

输入: n = 3
输出: ["((()))", "(())()", "()(())", "()()()", "()()()"]

```

解题思路：本题可以使用 **回溯算法 (Backtracking)**。在构建括号字符串的过程中，需要保证括号始终有效的，因此需要记录当前已经使用的左括号数量 `left` 和右括号数量 `right`。规则是：左括号数量不能超过 n ；右括号数量不能超过左括号数量，否则会形成非法括号。当字符串长度达到 $2 * n$ 时，说明形成了一个完整的合法括号组合，将其加入结果。通过不断尝试添加 '(' 或 ')' 并回溯，就可以生成所有可能的合法组合。

复杂度分析：时间复杂度为 $O(4^n / \sqrt{n})$ ，这是生成第 n 个 **卡特兰数** 数量的括号组合的复杂度。空间复杂度为 $O(n)$ ，主要来自递归调用栈和当前构建的字符串（不包括结果存储）。

Python代码：

```

from typing import List

class Solution:
    def generateParenthesis(self, n: int) -> List[str]:
        res = []
        path = []

        def backtrack(left, right):
            if len(path) == 2 * n:
                res.append("".join(path))
                return

            if left < n:
                path.append('(')
                backtrack(left + 1, right)
                path.pop()

            if right < left:
                path.append(')')
                backtrack(left, right + 1)
                path.pop()

        backtrack(0, 0)
        return res

```



```

if left < n:
    path.append("(")
    backtrack(left + 1, right)
    path.pop()

if right < left:
    path.append(")")
    backtrack(left, right + 1)
    path.pop()

backtrack(0, 0)
return res

```

6. 单词搜索（中等）

给定一个 $m \times n$ 二维字符网格 `board` 和一个字符串单词 `word`。如果 `word` 存在于网格中，返回 `true`；否则，返回 `false`。单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

示例：

A	B	C	E
S	F	C	S
A	D	E	E

输入: `board = [['A','B','C','E'],['S','F','C','S'],['A','D','E','E']]`, `word = "ABCCED"`

输出: `true`

解题思路：本题可以使用 **回溯 + 深度优先搜索（DFS）**。遍历网格中的每个位置作为起点，如果该位置字符与 `word` 的第一个字符匹配，则开始 DFS 搜索。每一步检查当前位置是否匹配 `word[index]`，然后向上下左右四个方向继续搜索下一个字符。为了避免重复使用同一个单元格，可以在访问时临时将该位置标记为已访问（例如改为 `'#'`），递归结束后再恢复原字符。如果成功匹配到 `word` 的最后一个字符，则返回 `True`。

复杂度分析：时间复杂度为 $O(m \times n \times 4^L)$ ，其中 $m \times n$ 是网格大小， L 是单词长度，每个字符最多向四个方向搜索。空间复杂度为 $O(L)$ ，主要来自递归调用栈的深度。

Python代码：

```

from typing import List

class Solution:
    def exist(self, board: List[List[str]], word: str) -> bool:
        m, n = len(board), len(board[0])

        def dfs(i, j, k):
            if k == len(word):
                return True

            # 上下左右四个方向
            directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
            for di, dj in directions:
                ni, nj = i + di, j + dj
                if 0 <= ni < m and 0 <= nj < n and board[ni][nj] == word[k]:
                    board[ni][nj] = '#'
                    if dfs(ni, nj, k + 1):
                        return True
                    board[ni][nj] = word[k]

            return False

        for i in range(m):
            for j in range(n):
                if board[i][j] == word[0]:
                    board[i][j] = '#'
                    if dfs(i, j, 1):
                        return True
                    board[i][j] = word[0]

        return False

```

```

        if i < 0 or i >= m or j < 0 or j >= n or board[i][j] != word[k]:
            return False

        temp = board[i][j]
        board[i][j] = "#" # 标记已访问

        found = (
            dfs(i+1, j, k+1) or
            dfs(i-1, j, k+1) or
            dfs(i, j+1, k+1) or
            dfs(i, j-1, k+1)
        )

        board[i][j] = temp # 回溯恢复

        return found

    for i in range(m):
        for j in range(n):
            if dfs(i, j, 0):
                return True

    return False

```

7. 分割回文串（中等）

给你一个字符串 `s`，请你将 `s` 分割成一些子串，使每个子串都是 **回文串**。返回 `s` 所有可能的分割方案。

示例：

```

输入：s = "aab"
输出：[["a","a","b"],["aa","b"]]

```

解题思路：本题可以使用 **回溯算法（Backtracking）**。从字符串的起始位置开始，枚举所有可能的子串，如果当前子串是回文串，则加入当前路径 `path`，然后递归处理剩余的字符串部分。当起始位置到达字符串末尾时，说明当前路径形成了一种合法分割方案，将其加入结果。判断回文串可以通过双指针或直接使用字符串反转。通过不断尝试不同的切割位置并回溯，就能得到所有可能的回文分割方案。

复杂度分析：时间复杂度为 $O(n \times 2^n)$ ，因为字符串的切割方式最多为 2^n 种，每次需要判断子串是否为回文。空间复杂度为 $O(n)$ ，主要来自递归调用栈和当前路径 `path`（不包括结果存储）。

Python代码：

```

from typing import List

class Solution:
    def partition(self, s: str) -> List[List[str]]:
        res = []
        path = []

        def is_palindrome(sub):

```

```

        return sub == sub[::-1]

    def backtrack(start):
        if start == len(s):
            res.append(path[:])
            return

        for end in range(start + 1, len(s) + 1):
            substring = s[start:end]
            if is_palindrome(substring):
                path.append(substring)
                backtrack(end)
                path.pop()

    backtrack(0)
    return res

```

8. N皇后（困难）

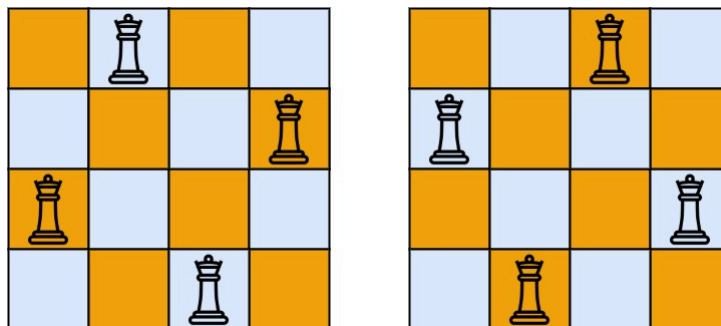
按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

n 皇后问题 研究的是如何将 n 个皇后放置在 $n \times n$ 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 n ，返回所有不同的 **n 皇后问题** 的解决方案。

每一种解法包含一个不同的 **n 皇后问题** 的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例：



输入：n = 4

输出：[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]

解释：如上图所示，4 皇后问题存在两个不同的解法。

解题思路： 使用回溯算法逐行尝试放置皇后。在放置每个皇后时，需要保证该列、主对角线（row - col）和副对角线（row + col）没有其他皇后冲突。若当前行放置成功，则递归下一行；如果到最后一行都放置成功，则保存当前棋盘状态为一种解。回溯时撤销选择，继续尝试其他列。

复杂度分析： 时间复杂度 $O(N!)$ ，因为每行最多 N 个位置可选，但由于冲突剪枝，实际搜索空间远小于 $N!$ ；空间复杂度 $O(N^2)$ ，递归栈深度为 N ，棋盘占用 $O(N^2)$ 空间。

Python 代码：

```

from typing import List

```

```

class Solution:
    def solveNQueens(self, n: int) -> List[List[str]]:
        res = []
        board = [['.'] * n for _ in range(n)]
        col = set()
        diag1 = set()
        diag2 = set()

        def backtrack(row: int):
            if row == n:
                res.append([''.join(r) for r in board])
                return
            for c in range(n):
                if c in col or (row - c) in diag1 or (row + c) in diag2:
                    continue
                board[row][c] = 'Q'
                col.add(c)
                diag1.add(row - c)
                diag2.add(row + c)
                backtrack(row + 1)
                board[row][c] = '.'
                col.remove(c)
                diag1.remove(row - c)
                diag2.remove(row + c)

        backtrack(0)
        return res

```

十一. 二分查找

1. 搜索插入位置（简单）

给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。

请必须使用时间复杂度为 $O(\log n)$ 的算法。

示例：

输入：nums = [1,3,5,6]，target = 5
输出：2

解题思路：因为数组有序，可以使用二分查找。每次比较中间元素与目标值，如果中间元素小于目标值，则搜索右半部分，否则搜索左半部分。最终循环结束时，左指针即为目标值应该插入的位置（如果目标值存在则指向其索引，如果不存在则是插入位置）。

复杂度分析：时间复杂度 $O(\log n)$ ，因为每次循环将搜索范围减半；空间复杂度 $O(1)$ ，只使用了常数级额外变量。

Python 代码：

```

from typing import List

```

```
class Solution:
    def searchInsert(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                left = mid + 1
            else:
                right = mid - 1
        return left
```

2. 搜索二维矩阵（中等）

给你一个满足下述两条属性的 $m \times n$ 整数矩阵：

- 每行中的整数从左到右按非严格递增顺序排列。
- 每行的第一个整数大于前一行的最后一个整数。

给你一个整数 `target`，如果 `target` 在矩阵中，返回 `true`；否则，返回 `false`。

示例：

1	3	5	7
10	11	16	20
23	30	34	60

输入: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
输出: true

解题思路：因为矩阵每行升序且每行首元素大于上一行的尾元素，可以将整个矩阵看作一个排好序的一维数组，利用二分查找搜索目标值。计算中间元素时用 `matrix[mid // n][mid % n]` 映射到二维位置。

复杂度分析：时间复杂度 $O(\log(mn))$ ，二分查找遍历 $\log(mn)$ 步；空间复杂度 $O(1)$ ，只使用常数额外变量。

Python 代码：

```
from typing import List

class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False
        m, n = len(matrix), len(matrix[0])
        left, right = 0, m * n - 1
        while left <= right:
```

```

        mid = (left + right) // 2
        mid_val = matrix[mid // n][mid % n]
        if mid_val == target:
            return True
        elif mid_val < target:
            left = mid + 1
        else:
            right = mid - 1
    return False

```

3. 在排序数组中查找元素的第一个和最后一个位置（中等）

给你一个按照非递减顺序排列的整数数组 `nums`，和一个目标值 `target`。请你找出给定目标值在数组中的开始位置和结束位置。如果数组中不存在目标值 `target`，返回 `[-1, -1]`。你必须设计并实现时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例：

输入: `nums = [5,7,7,8,8,10]`, `target = 8`
 输出: `[3,4]`

解题思路：利用二分查找分别寻找目标值的 **左边界**（第一个出现的位置）和 **右边界**（最后一个出现的位置）。在查找左边界时，如果 `nums[mid] >= target`，右边界缩到 `mid-1`；在查找右边界时，如果 `nums[mid] <= target`，左边界移动到 `mid+1`。这样可以在 $O(\log n)$ 时间内定位目标值的区间。

复杂度分析：时间复杂度 $O(\log n)$ ，因为左右边界各用一次二分查找；空间复杂度 $O(1)$ ，仅使用常数额外变量。

Python 代码：

```

from typing import List

class Solution:
    def searchRange(self, nums: List[int], target: int) -> List[int]:
        def findLeft(nums, target):
            left, right = 0, len(nums) - 1
            idx = -1
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] >= target:
                    right = mid - 1
                else:
                    left = mid + 1
                if nums[mid] == target:
                    idx = mid
            return idx

        def findRight(nums, target):
            left, right = 0, len(nums) - 1
            idx = -1
            while left <= right:
                mid = (left + right) // 2
                if nums[mid] <= target:

```

```

        left = mid + 1
    else:
        right = mid - 1
    if nums[mid] == target:
        idx = mid
    return idx

return [findLeft(nums, target), findRight(nums, target)]

```

4. 搜索旋转排序数组（中等）

整数数组 `nums` 按升序排列，数组中的值 **互不相同**。在传递给函数之前，`nums` 在预先未知的某个下标 `k` ($0 \leq k < \text{nums.length}$) 上进行了 **向左旋转**，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]`（下标 **从 0 开始** 计数）。例如，`[0,1,2,4,5,6,7]` 下标 3 上向左旋转后可能变为 `[4,5,6,7,0,1,2]`。给你 **旋转后** 的数组 `nums` 和一个整数 `target`，如果 `nums` 中存在这个目标值 `target`，则返回它的下标，否则返回 `-1`。你必须设计一个时间复杂度 $O(\log n)$ 算法解决此问题。

示例：

输入: `nums = [4,5,6,7,0,1,2]`, `target = 0`
 输出: 4

解题思路：由于数组是旋转排序数组，可以利用 **修改版二分查找**。在每次二分时，通过比较 `nums[mid]` 与 `nums[left]` 判断哪一半是有序的，然后根据 `target` 是否在有序区间内来缩小搜索范围。这样可以在 $O(\log n)$ 时间内定位目标值。

复杂度分析：时间复杂度 $O(\log n)$ ，每次循环将搜索范围减半；空间复杂度 $O(1)$ ，只使用常数变量存储索引。

Python 代码：

```

from typing import List

class Solution:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1

        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid

            # 判断左半部分是否有序
            if nums[left] <= nums[mid]:
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            # 右半部分有序
            else:
                if nums[mid] < target <= nums[right]:
                    left = mid + 1

```

```
        else:
            right = mid - 1

    return -1
```

5. 寻找旋转排序数组中的最小值（中等）

已知一个长度为 n 的数组，预先按照升序排列，经由 1 到 n 次 **旋转** 后，得到输入数组。例如，原数组 `nums = [0,1,2,4,5,6,7]` 在变化后可能得到：

- 若旋转 4 次，则可以得到 `[4,5,6,7,0,1,2]`
- 若旋转 7 次，则可以得到 `[0,1,2,4,5,6,7]`

注意，数组 `[a[0], a[1], a[2], ..., a[n-1]]` **旋转一次** 的结果为数组 `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`。给你一个元素值 **互不相同** 的数组 `nums`，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的 **最小元素**。你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例：

输入：`nums = [3,4,5,1,2]`
输出：1
解释：原数组为 `[1,2,3,4,5]`，旋转 3 次得到输入数组。

解题思路：数组原本是升序排列，但经过旋转后会形成两段有序区间，最小值一定出现在两段的分界点。可以使用二分查找，通过比较 `nums[mid]` 与 `nums[right]` 来判断最小值所在的位置。如果 `nums[mid] > nums[right]`，说明最小值在右半部分，因此移动左指针 `left = mid + 1`；如果 `nums[mid] < nums[right]`，说明最小值在左半部分（包含 `mid`），因此移动右指针 `right = mid`。不断缩小区间直到 `left == right`，此时该位置的元素就是数组中的最小值。

复杂度分析：时间复杂度为 $O(\log n)$ ，因为每次通过二分查找将搜索区间缩小一半；空间复杂度为 $O(1)$ ，只使用了常数级额外变量来记录指针位置。

Python代码：

```
from typing import List

class Solution:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1

        while left < right:
            mid = (left + right) // 2

            if nums[mid] > nums[right]:
                left = mid + 1
            else:
                right = mid

        return nums[left]
```


6. 寻找两个正序数组的中位数（困难）

给定两个大小分别为 m 和 n 的正序（从小到大）数组 `nums1` 和 `nums2`。请你找出并返回这两个正序数组的 **中位数**。算法的时间复杂度应该为 $O(\log(m+n))$ 。

示例：

输入：nums1 = [1,3]，nums2 = [2]

输出：2.00000

解释：合并数组 = [1,2,3]，中位数 2

解题思路：由于两个数组都是升序排列，并且要求时间复杂度为 $O(\log(m+n))$ ，不能直接合并数组。可以使用二分查找对较短的数组进行分割，使两个数组被分成左右两部分，并满足左部分所有元素都小于等于右部分元素，同时左右两部分元素数量相等（或左边多一个）。设 i 为 `nums1` 的分割位置， j 为 `nums2` 的分割位置，使 $i + j = (m+n+1)//2$ 。当满足 `nums1[i-1] <= nums2[j]` 且 `nums2[j-1] <= nums1[i]` 时说明分割正确，此时可以根据总长度奇偶计算中位数。如果不满足条件，则调整二分查找的范围继续寻找合适的分割位置。

复杂度分析：时间复杂度为 $O(\log(\min(m,n)))$ ，因为只在较短的数组上进行二分查找；空间复杂度为 $O(1)$ ，只使用了常数级变量存储指针和边界值。

Python代码：

```
from typing import List

class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1

        m, n = len(nums1), len(nums2)
        left, right = 0, m

        while left <= right:
            i = (left + right) // 2
            j = (m + n + 1) // 2 - i

            nums1_left = float('-inf') if i == 0 else nums1[i-1]
            nums1_right = float('inf') if i == m else nums1[i]

            nums2_left = float('-inf') if j == 0 else nums2[j-1]
            nums2_right = float('inf') if j == n else nums2[j]

            if nums1_left <= nums2_right and nums2_left <= nums1_right:
                if (m + n) % 2 == 1:
                    return max(nums1_left, nums2_left)
                else:
                    return (max(nums1_left, nums2_left) + min(nums1_right,
                                                                nums2_right)) / 2
            elif nums1_left > nums2_right:
                right = i - 1
            else:
                left = i + 1
```

十二. 栈

1. 有效的括号（简单）

给定一个只包括 '('、')'、'{'、'}'、'['、']' 的字符串 `s`，判断字符串是否有效。有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

示例：

输入：s = "()"

输出：true

解题思路：使用栈来解决括号匹配问题。遍历字符串，如果遇到左括号（'（'、'['、'{'）就将其压入栈中；如果遇到右括号，则检查栈顶元素是否为对应的左括号，如果不是或者栈为空，则说明括号不匹配，返回 `False`。如果匹配成功则将栈顶元素弹出。遍历完成后，如果栈为空说明所有括号都正确匹配，否则说明仍有未匹配的左括号。

复杂度分析：时间复杂度为 $O(n)$ ，需要遍历字符串一次；空间复杂度为 $O(n)$ ，在最坏情况下（全部为左括号）需要栈存储所有字符。

Python代码：

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        mapping = {
            ')': '(',
            ']': '[',
            '}': '{'
        }

        for ch in s:
            if ch in mapping:
                if not stack or stack[-1] != mapping[ch]:
                    return False
                stack.pop()
            else:
                stack.append(ch)

        return not stack
```

2. 最小栈（中等）

设计一个支持 `push` , `pop` , `top` 操作，并能在常数时间内检索到最小元素的栈。

实现 `MinStack` 类:

- `MinStack()` 初始化堆栈对象。
- `void push(int val)` 将元素`val`推入堆栈。
- `void pop()` 删除堆栈顶部的元素。
- `int top()` 获取堆栈顶部的元素。
- `int getMin()` 获取堆栈中的最小元素。

示例:

输入:

```
["MinStack","push","push","push","getMin","pop","top","getMin"]  
[[],[-2],[0],[-3],[[],[],[],[]]]
```

输出:

```
[null,null,null,null,-3,null,0,-2]
```

解释:

```
MinStack minStack = new MinStack();  
minStack.push(-2);  
minStack.push(0);  
minStack.push(-3);  
minStack.getMin();   --> 返回 -3.  
minStack.pop();  
minStack.top();       --> 返回 0.  
minStack.getMin();    --> 返回 -2.
```

解题思路: 为了在 $O(1)$ 时间获取最小值，可以使用 **两个栈**：一个普通栈 `stack` 用来存储所有元素，另一个最小栈 `min_stack` 用来记录当前的最小值。当执行 `push` 时，将元素压入 `stack`，同时如果 `min_stack` 为空或当前元素小于等于 `min_stack` 栈顶，则也压入 `min_stack`。当执行 `pop` 时，如果弹出的元素等于 `min_stack` 栈顶，则同时弹出 `min_stack` 的栈顶。这样 `min_stack` 的栈顶始终保存当前栈中的最小值，因此 `getMin()` 只需返回 `min_stack` 栈顶即可。

复杂度分析: 所有操作 `push`、`pop`、`top`、`getMin` 的时间复杂度都是 $O(1)$ ，因为都只涉及栈顶操作；空间复杂度为 $O(n)$ ，最坏情况下两个栈都需要存储所有元素。

Python代码:

```
class MinStack:  
  
    def __init__(self):  
        self.stack = []  
        self.min_stack = []  
  
    def push(self, val: int) -> None:  
        self.stack.append(val)  
        if not self.min_stack or val <= self.min_stack[-1]:  
            self.min_stack.append(val)
```

```

def pop(self) -> None:
    val = self.stack.pop()
    if val == self.min_stack[-1]:
        self.min_stack.pop()

def top(self) -> int:
    return self.stack[-1]

def getMin(self) -> int:
    return self.min_stack[-1]

```

3. 字符串解码（中等）

给定一个经过编码的字符串，返回它解码后的字符串。编码规则为: $k[\text{encoded_string}]$ ，表示其中方括号内部的 `encoded_string` 正好重复 k 次。注意 k 保证为正整数。你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 k ，例如不会出现像 `3a` 或 `2[4]` 的输入。测试用例保证输出的长度不会超过 105。

示例：

```

输入：s = "3[a]2[bc]"
输出："aaabcbc"

```

解题思路：使用栈来处理嵌套的括号结构。遍历字符串时，如果遇到数字就计算当前重复次数 k ；如果遇到 `[`，说明进入新的编码段，将当前的字符串和重复次数压入栈中，然后重置当前字符串和计数；如果遇到 `]`，说明当前编码段结束，从栈中弹出之前保存的字符串和重复次数，将当前字符串重复 k 次并拼接回去；如果是普通字母，则直接加入当前字符串。遍历结束后得到的字符串就是解码后结果。

复杂度分析：时间复杂度为 $O(n)$ ，其中 n 为字符串长度，每个字符只遍历一次；空间复杂度为 $O(n)$ ，用于栈存储中间状态以及构造最终字符串。

Python代码：

```

class Solution:
    def decodeString(self, s: str) -> str:
        stack = []
        cur_num = 0
        cur_str = ""

        for ch in s:
            if ch.isdigit():
                cur_num = cur_num * 10 + int(ch)
            elif ch == '[':
                stack.append((cur_str, cur_num))
                cur_str = ""
                cur_num = 0
            elif ch == ']':
                prev_str, num = stack.pop()
                cur_str = prev_str + num * cur_str
            else:
                cur_str += ch

```

```
return cur_str
```

4. 每日温度 (中等)

给定一个整数数组 `temperatures`，表示每天的温度，返回一个数组 `answer`，其中 `answer[i]` 是指对于第 `i` 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

示例：

```
输入: temperatures = [73,74,75,71,69,72,76,73]
输出: [1,1,4,2,1,1,0,0]
```

解题思路：使用 **单调栈（单调递减栈）** 来解决问题。栈中存储的是温度数组的下标，并保持栈中对应的温度值从栈底到栈顶递减。遍历数组时，如果当前温度大于栈顶下标对应的温度，说明找到了栈顶那一天之后的更高温度，此时不断弹出栈顶元素，并计算天数差 `i - index` 作为答案。然后将当前下标压入栈中。遍历结束后，栈中剩余的下标表示之后都没有更高温度，对应答案保持为 `0`。

复杂度分析：时间复杂度为 $O(n)$ ，每个元素最多进栈和出栈一次；空间复杂度为 $O(n)$ ，用于存储栈以及结果数组。

Python代码：

```
from typing import List

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        n = len(temperatures)
        res = [0] * n
        stack = []

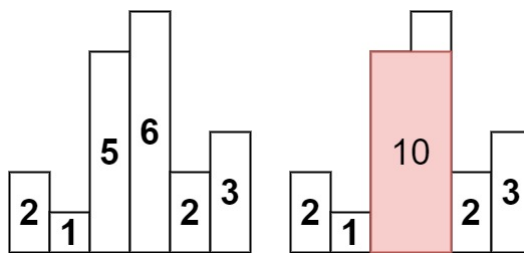
        for i in range(n):
            while stack and temperatures[i] > temperatures[stack[-1]]:
                idx = stack.pop()
                res[idx] = i - idx
            stack.append(i)

        return res
```

5. 柱状图中最大的矩形 (困难)

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例：



输入: `heights = [2,1,5,6,2,3]`

输出: 10

解释: 最大的矩形为图中红色区域, 面积为 10

解题思路: 可以使用 **单调栈 (递增栈)** 来解决。栈中存储的是柱子的下标, 并保持对应的高度递增。当遍历到当前柱子时, 如果当前高度小于栈顶柱子的高度, 说明以栈顶柱子为高的最大矩形已经确定, 此时可以弹出栈顶元素并计算面积。矩形的高度为被弹出的柱子高度, 宽度为当前下标与新的栈顶下标之间的距离。为了保证所有柱子都能被计算, 可以在数组前后各添加一个高度为 0 的柱子作为哨兵, 这样遍历结束时栈会被完全清空, 从而计算出所有可能的矩形面积。

复杂度分析: 时间复杂度为 $O(n)$, 每个柱子最多进栈和出栈一次; 空间复杂度为 $O(n)$, 用于存储栈。

Python代码:

```
from typing import List

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        heights = [0] + heights + [0]
        stack = []
        max_area = 0

        for i in range(len(heights)):
            while stack and heights[i] < heights[stack[-1]]:
                h = heights[stack.pop()]
                w = i - stack[-1] - 1
                max_area = max(max_area, h * w)
            stack.append(i)

        return max_area
```

6. 数组中的第K个最大元素 (中等)

给定整数数组 `nums` 和整数 `k`, 请返回数组中第 `**k**` 个最大的元素。请注意, 你需要找的是数组排序后的第 `k` 个最大的元素, 而不是第 `k` 个不同的元素。你必须设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例:

输入: `[3,2,1,5,6,4]`, `k = 2`

输出: 5

解题思路：可以使用 **快速选择算法 (Quick Select)** 来在平均 $O(n)$ 时间内找到第 k 个最大元素。思路类似快速排序：随机选择一个 pivot，将数组划分为大于 pivot、等于 pivot、和小于 pivot 的三部分。根据划分结果判断第 k 个最大元素在哪一部分，然后递归到那一部分继续寻找。为了方便处理“第 k 个最大”，可以把问题转化为寻找第 $(\text{len}(\text{nums}) - k)$ 个最小元素。相比直接排序，快速选择不需要对整个数组排序，效率更高。

复杂度分析：平均时间复杂度为 $O(n)$ ，最坏情况为 $O(n^2)$ （可通过随机化 pivot 避免）；空间复杂度为 $O(1)$ 或递归栈 $O(\log n)$ 。

Python代码：

```
from typing import List
import random

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        def quick_select(left, right, index):
            pivot = nums[random.randint(left, right)]
            l, r = left, right
            while l <= r:
                while l <= r and nums[l] < pivot:
                    l += 1
                while l <= r and nums[r] > pivot:
                    r -= 1
                if l <= r:
                    nums[l], nums[r] = nums[r], nums[l]
                    l += 1
                    r -= 1
            if left + k - 1 <= r:
                return quick_select(left, r, index)
            if l - 1 < left + k - 1:
                return quick_select(l, right, index)
            return pivot

        # 转换为寻找第 len(nums) - k 个最小元素
        target = len(nums) - k
        left, right = 0, len(nums) - 1
        while True:
            pivot_index = random.randint(left, right)
            pivot = nums[pivot_index]
            # 三路划分
            low, high = left, right
            i = left
            while i <= high:
                if nums[i] < pivot:
                    nums[low], nums[i] = nums[i], nums[low]
                    low += 1
                    i += 1
                elif nums[i] > pivot:
                    nums[high], nums[i] = nums[i], nums[high]
                    high -= 1
                else:
                    i += 1
            if target < low:
                right = low - 1
```

```
elif target > high:
    left = high + 1
else:
    return pivot
```

7. 前K个高频元素 (中等)

给你一个整数数组 `nums` 和一个整数 `k`，请你返回其中出现频率前 `k` 高的元素。你可以按 **任意顺序** 返回答案。

示例：

输入: `nums = [1,1,1,2,2,3]`, `k = 2`
输出: `[1,2]`

解题思路：可以先使用哈希表统计每个元素的出现频率，然后利用 **最小堆 (heap)** 或 **快速选择 (Quick Select)** 找出前 `k` 个高频元素。方法一是构建一个长度为 `k` 的最小堆，遍历哈希表，将频率最大的 `k` 个元素保留在堆中；方法二是把哈希表转换为列表，根据频率用快速选择找到第 `k` 大的频率，从而获取所有频率大于等于该值的元素。对于大数据量时，时间复杂度更优。

复杂度分析：哈希表统计频率： $O(n)$ ，`n` 为数组长度。最小堆操作： $O(n \log k)$ 或者快速选择平均 $O(n)$ 。空间复杂度： $O(n)$ 用于存储频率计数

Python代码：

```
from typing import List
from collections import Counter
import heapq

class Solution:
    def topKFrequent(self, nums: List[int], k: int) -> List[int]:
        # 统计频率
        count = Counter(nums)
        # 用最小堆存储前 k 个元素
        heap = []
        for num, freq in count.items():
            if len(heap) < k:
                heapq.heappush(heap, (freq, num))
            else:
                if freq > heap[0][0]:
                    heapq.heappushpop(heap, (freq, num))
        return [num for freq, num in heap]
```

8. 数据流的中位数 (困难)

中位数是有序整数列表中的中间值。如果列表的大小是偶数，则没有中间值，中位数是两个中间值的平均值。

- 例如 `arr = [2,3,4]` 的中位数是 `3`。
- 例如 `arr = [2,3]` 的中位数是 $(2 + 3) / 2 = 2.5$ 。

实现 MedianFinder 类:

- `MedianFinder()` 初始化 `MedianFinder` 对象。
- `void addNum(int num)` 将数据流中的整数 `num` 添加到数据结构中。
- `double findMedian()` 返回到目前为止所有元素的中位数。与实际答案相差 10^{-5} 以内的答案将被接受。

示例:

输入

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]  
[[], [1], [2], [], [3], []]
```

输出

```
[null, null, null, 1.5, null, 2.0]
```

解释

```
MedianFinder medianFinder = new MedianFinder();  
medianFinder.addNum(1);    // arr = [1]  
medianFinder.addNum(2);    // arr = [1, 2]  
medianFinder.findMedian(); // 返回 1.5 ((1 + 2) / 2)  
medianFinder.addNum(3);    // arr[1, 2, 3]  
medianFinder.findMedian(); // return 2.0
```

解题思路: 维护两个堆来实现动态获取中位数: 一个 **最大堆** 保存较小一半的元素, 一个 **最小堆** 保存较大一半的元素。插入新数时, 将其放入合适的堆, 并保持两个堆大小平衡 (最大堆最多比最小堆多一个元素)。中位数即为两堆顶元素的平均 (总数为偶数) 或最大堆堆顶 (总数为奇数)。这种方法保证每次插入和获取中位数都高效。

复杂度分析: 插入操作 `addNum`: $O(\log n)$ 。(堆插入和调整) 获取中位数 `findMedian`: $O(1)$ 。空间复杂度: $O(n)$ 用于存储堆中的元素

Python代码:

```
import heapq  
  
class MedianFinder:  
    def __init__(self):  
        # 最大堆, 存储较小的一半 (Python heapq 默认最小堆, 用负数模拟最大堆)  
        self.max_heap = []  
        # 最小堆, 存储较大的一半  
        self.min_heap = []  
  
    def addNum(self, num: int) -> None:  
        # 先入最大堆  
        heapq.heappush(self.max_heap, -num)  
        # 保证最大堆顶 <= 最小堆顶  
        heapq.heappush(self.min_heap, -heapq.heappop(self.max_heap))  
        # 调整堆大小, 最大堆可多一个元素  
        if len(self.min_heap) > len(self.max_heap):  
            heapq.heappush(self.max_heap, -heapq.heappop(self.min_heap))  
  
    def findMedian(self) -> float:  
        if len(self.max_heap) > len(self.min_heap):  
            return -self.max_heap[0]
```

```
else:
    return (-self.max_heap[0] + self.min_heap[0]) / 2
```

十三. 贪心算法

1. 买卖股票的最佳时机（简单）

给定一个数组 `prices`，它的第 `i` 个元素 `prices[i]` 表示一支给定股票第 `i` 天的价格。你只能选择**某一天** 买入这只股票，并选择在**未来的某一个不同的日子** 卖出该股票。设计一个算法来计算你能获取的最大利润。返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 `0`。

示例：

输入：[7,1,5,3,6,4]

输出：5

解释：在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5。

注意利润不能是 7-1 = 6，因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。

解题思路：维护一个变量记录到目前为止的最低买入价格，然后遍历数组，对于每一天的价格计算如果今天卖出能获得的利润，并更新最大利润。这样只需一次遍历即可得到最大收益。核心思想是**找到最小买入点与未来最大卖出点的差值**。

复杂度分析：时间复杂度：O(n)，只需遍历一次价格数组。空间复杂度：O(1)，只需常数额外空间存储当前最小价格和最大利润。

Python代码：

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        if not prices:
            return 0

        min_price = float('inf')
        max_profit = 0

        for price in prices:
            min_price = min(min_price, price)
            max_profit = max(max_profit, price - min_price)

        return max_profit
```

2. 跳跃游戏（中等）

给你一个非负整数数组 `nums`，你最初位于数组的**第一个下标**。数组中的每个元素代表你在该位置可以跳跃的最大长度。判断你是否能够到达最后一个下标，如果可以，返回 `true`；否则，返回 `false`。

示例：

输入: `nums = [2,3,1,1,4]`

输出: `true`

解释: 可以先跳 1 步, 从下标 0 到达下标 1, 然后再从下标 1 跳 3 步到达最后一个下标。

解题思路: 我们可以维护一个变量 `farthest` 表示目前能够到达的最远位置, 然后从左向右遍历数组, 对于每个位置 `i`, 如果 `i` 超过了 `farthest`, 说明当前位置不可达, 直接返回 `False`; 否则更新 `farthest = max(farthest, i + nums[i])`。遍历结束后, 如果 `farthest` 大于等于最后一个下标, 则可以到达最后位置, 返回 `True`。这种贪心策略保证每一步都尽量向前跳, 从而判断是否可达。

复杂度分析: 时间复杂度: $O(n)$, 只需要遍历一次数组。空间复杂度: $O(1)$, 只使用常数额外空间存储最远可达位置。

Python代码:

```
class Solution:
    def canJump(self, nums: list[int]) -> bool:
        farthest = 0
        n = len(nums)

        for i in range(n):
            if i > farthest:
                return False
            farthest = max(farthest, i + nums[i])

        return True
```

3. 跳跃游戏II (中等)

给定一个长度为 `n` 的 **0 索引** 整数数组 `nums`。初始位置在下标 0。每个元素 `nums[i]` 表示从索引 `i` 向后跳转的最大长度。换句话说, 如果你在索引 `i` 处, 你可以跳转到任意 `(i + j)` 处:

- `0 <= j <= nums[i]` 且
- `i + j < n`

返回到达 `n - 1` 的最小跳跃次数。测试用例保证可以到达 `n - 1`。

示例:

输入: `nums = [2,3,1,1,4]`

输出: 2

解释: 跳到最后一个位置的最小跳跃数是 2。

从下标为 0 跳到下标为 1 的位置, 跳 1 步, 然后跳 3 步到达数组的最后一个位置。

解题思路: 这道题是跳跃游戏的扩展, 要求最少跳跃次数。可以使用贪心策略: 维护两个变量 `end` 表示当前跳跃的覆盖范围的右边界, `farthest` 表示在当前覆盖范围内能到达的最远位置。从左向右遍历数组, 每次更新 `farthest = max(farthest, i + nums[i])`, 当遍历到 `i == end` 时, 说明必须进行一次跳跃, 将 `end` 更新为 `farthest`, 并增加跳跃次数 `jumps`。遍历结束即可得到到达最后一个位置的最少跳跃次数。

复杂度分析: 时间复杂度: $O(n)$, 只需要遍历一次数组。空间复杂度: $O(1)$, 只使用常数额外变量存储覆盖范围和跳跃次数。

Python代码：

```
class Solution:
    def jump(self, nums: list[int]) -> int:
        n = len(nums)
        jumps = 0
        end = 0
        farthest = 0

        for i in range(n - 1): # 不需要考虑最后一个位置
            farthest = max(farthest, i + nums[i])

            if i == end: # 到达当前跳跃覆盖范围的末尾，需要跳跃
                jumps += 1
                end = farthest

        return jumps
```

4. 划分字母区间（中等）

给你一个字符串 `s`。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。例如，字符串 `"ababcc"` 能够被分为 `["abab", "cc"]`，但类似 `["aba", "bcc"]` 或 `["ab", "ab", "cc"]` 的划分是非法的。注意，划分结果需要满足：将所有划分结果按顺序连接，得到的字符串仍然是 `s`。返回一个表示每个字符串片段的长度的列表。

示例：

输入：s = "ababcbacadeefegdehijhklj"

输出：[9,7,8]

解释：

划分结果为 "ababcbaca"、"defegde"、"hijhklj"。

每个字母最多出现在一个片段中。

像 "ababcbacadeefegde"，"hijhklj" 这样的划分是错误的，因为划分的片段数较少。

解题思路：我们可以用贪心的方法解决这个问题。首先记录每个字母在字符串中最后出现的位置，然后从左向右遍历字符串，维护当前片段的右边界 `end` 为遍历过字母的最后出现位置的最大值。当遍历到下标等于当前片段右边界时，说明这个片段可以结束，将片段长度加入结果列表，并更新下一个片段的起点。这样可以保证每个字母最多出现在一个片段中，同时片段数量尽可能多。

复杂度分析：时间复杂度：O(n)，需要遍历字符串两次（一次记录每个字母最后出现位置，一次划分片段）。空间复杂度：O(1)，因为字母数量有限（26 个小写字母），额外存储每个字母的最后出现位置即可。

Python代码：

```
class Solution:
    def partitionLabels(self, s: str) -> list[int]:
        # 记录每个字母最后出现的位置
        last_index = {c: i for i, c in enumerate(s)}

        res = []
        start = 0
```

```

end = 0

for i, c in enumerate(s):
    end = max(end, last_index[c]) # 更新当前片段右边界
    if i == end: # 到达片段末尾
        res.append(end - start + 1)
        start = i + 1 # 下一片段起点

return res

```

十四. 动态规划

1. 爬楼梯（简单）

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶呢？

示例：

```

输入：n = 2
输出：2
解释：有两种方法可以爬到楼顶。
1. 1 阶 + 1 阶
2. 2 阶

```

解题思路：这道题是典型的动态规划问题。设 $dp[i]$ 为到达第 i 阶楼梯的方法数，由于每次可以爬 1 阶或 2 阶，因此到达第 i 阶的方法数等于到达 $i-1$ 阶的方法数加上到达 $i-2$ 阶的方法数，即 $dp[i] = dp[i-1] + dp[i-2]$ 。初始条件为 $dp[1] = 1$ 、 $dp[2] = 2$ 。这样自底向上计算 $dp[n]$ 即可得到答案。

复杂度分析：时间复杂度： $O(n)$ ，只需一次循环计算 dp 数组。空间复杂度： $O(1)$ ，可以只使用两个变量保存前两阶的状态，而不需要完整 dp 数组。

Python代码：

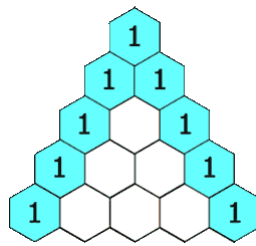
```

class Solution:
    def climbStairs(self, n: int) -> int:
        if n == 1:
            return 1
        a, b = 1, 2 # a: dp[i-2], b: dp[i-1]
        for _ in range(3, n + 1):
            a, b = b, a + b
        return b

```

2. 杨辉三角（简单）

给定一个非负整数 $numRows$ ，生成「杨辉三角」的前 $numRows$ 行。在「杨辉三角」中，每个数是它左上方和右上方的数的和。



示例:

输入: numRows = 5
输出: [[1], [1,1], [1,2,1], [1,3,3,1], [1,4,6,4,1]]

解题思路: 杨辉三角的每一行可以通过上一行计算得到。具体来说, 第 i 行的第 j 个元素等于上一行的第 $j-1$ 个元素与第 j 个元素之和 (边界元素默认为 1)。可以从第一行 `[1]` 开始, 依次生成每一行, 并添加到结果列表中。这样就能逐行构建整个三角形。

复杂度分析: 时间复杂度: $O(\text{numRows}^2)$, 每行需要计算的元素数量等于行号, 整体累加为 $1 + 2 + \dots + \text{numRows} = O(\text{numRows}^2)$ 。空间复杂度: $O(\text{numRows}^2)$, 用于存储整个杨辉三角的结果。

Python代码:

```
class Solution:
    def generate(self, numRows: int) -> list[list[int]]:
        res = []
        for i in range(numRows):
            row = [1] * (i + 1) # 每行首尾为1
            for j in range(1, i):
                row[j] = res[i-1][j-1] + res[i-1][j]
            res.append(row)
        return res
```

3. 打家劫舍 (中等)

你是一个专业的小偷, 计划偷窃沿街的房屋。每间房内都藏有一定的现金, 影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统, **如果两间相邻的房屋在同一晚上被小偷闯入, 系统会自动报警**。给定一个代表每个房屋存放金额的非负整数数组, 计算你 **不触动警报装置的情况下**, 一夜之内能够偷窃到的最高金额。

示例:

输入: [1,2,3,1]
输出: 4
解释: 偷窃 1 号房屋 (金额 = 1), 然后偷窃 3 号房屋 (金额 = 3)。
偷窃到的最高金额 = 1 + 3 = 4。

解题思路: 这道题是典型的动态规划问题。对第 i 个房屋, 有两种选择: 偷或者不偷。如果偷第 i 个房屋, 那么不能偷 $i-1$ 房屋, 最大金额为 $\text{dp}[i-2] + \text{nums}[i]$ 。如果不偷第 i 个房屋, 那么金额等于 $\text{dp}[i-1]$ 。因此状态转移方程为: $\text{dp}[i] = \max(\text{dp}[i-1], \text{dp}[i-2] + \text{nums}[i])$ 。可以用一个数组 `dp` 来存储前 i 个房屋能偷到的最大金额, 或者使用两个变量优化空间复杂度。

复杂度分析: 时间复杂度: $O(n)$, 遍历一次房屋数组即可。空间复杂度: $O(1)$ (优化后), 只需保存前两步的状态。

Python代码:

```
class Solution:
    def rob(self, nums: list[int]) -> int:
        if not nums:
            return 0
        n = len(nums)
        if n == 1:
            return nums[0]

        prev2, prev1 = 0, nums[0]
        for i in range(1, n):
            curr = max(prev1, prev2 + nums[i])
            prev2, prev1 = prev1, curr
        return prev1
```

4. 完全平方数 (中等)

给你一个整数 n ，返回和为 n 的完全平方数的最少数量。完全平方数是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

示例:

```
输入: n = 12
输出: 3
解释: 12 = 4 + 4 + 4
```

解题思路: 这道题可以用动态规划解决。设 $dp[i]$ 表示组成整数 i 所需的最少完全平方数数量。状态转移方程为:

$$dp[i] = \min(dp[i - j*j] + 1) \quad \text{for all } j \text{ where } j*j \leq i$$

也就是说，对于每个 i ，尝试所有不超过 i 的完全平方数 $j*j$ ，选取能够让 $dp[i]$ 最小的方案。初始化 $dp[0] = 0$ 。通过从 1 到 n 遍历更新 $dp[i]$ ，最终得到 $dp[n]$ 。

复杂度分析: 时间复杂度: $O(n * \sqrt{n})$ ，因为每个 i 最多尝试 $\sqrt{i}$ 个完全平方数。空间复杂度: $O(n)$ ，用于存储 dp 数组。

Python代码:

```
class Solution:
    def numSquares(self, n: int) -> int:
        dp = [float('inf')] * (n + 1)
        dp[0] = 0

        for i in range(1, n + 1):
            j = 1
            while j * j <= i:
                dp[i] = min(dp[i], dp[i - j*j] + 1)
                j += 1
        return dp[n]
```

5. 零钱兑换（中等）

给你一个整数数组 `coins`，表示不同面额的硬币；以及一个整数 `amount`，表示总金额。计算并返回可以凑成总金额所需的 **最少的硬币个数**。如果没有任何一种硬币组合能组成总金额，返回 `-1`。你可以认为每种硬币的数量是无限的。

示例：

```
输入: coins = [1, 2, 5], amount = 11
输出: 3
解释: 11 = 5 + 5 + 1
```

解题思路：这道题是经典的「完全背包问题」的变形。用动态规划求解，设 `dp[i]` 表示凑成金额 `i` 所需的最少硬币数。初始时 `dp[0] = 0`，其他 `dp[i]` 初始化为正无穷（表示无法凑成）。然后依次遍历每个金额 `i`，对每个硬币面额 `c`，如果 `i >= c`，就更新状态：

```
dp[i] = min(dp[i], dp[i - c] + 1)
```

最后 `dp[amount]` 就是答案，如果仍为无穷，说明无法凑出该金额，返回 `-1`。

复杂度分析：时间复杂度： $O(\text{amount} * \text{len}(\text{coins}))$ ，每个金额遍历所有硬币。空间复杂度： $O(\text{amount})$ ，用于存储 `dp` 数组。

Python代码：

```
class Solution:
    def coinChange(self, coins: list[int], amount: int) -> int:
        dp = [float('inf')] * (amount + 1)
        dp[0] = 0

        for i in range(1, amount + 1):
            for c in coins:
                if i >= c:
                    dp[i] = min(dp[i], dp[i - c] + 1)

        return dp[amount] if dp[amount] != float('inf') else -1
```

6. 单词拆分（中等）

给你一个字符串 `s` 和一个字符串列表 `wordDict` 作为字典。如果可以利用字典中出现的一个或多个单词拼接出 `s` 则返回 `true`。**注意：**不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例：

```
输入: s = "leetcode", wordDict = ["leet", "code"]
输出: true
解释: 返回 true 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。
```


解题思路：这是经典的动态规划问题。我们用一个布尔数组 `dp`，其中 `dp[i]` 表示字符串 `s[:i]` 是否可以由字典中的单词拼接而成。初始状态 `dp[0] = True`（空字符串可以被拼接）。然后遍历字符串的每个位置 `i`，再遍历字典中的单词 `word`，如果当前位置 `i` 之前有 `dp[i - len(word)] = True` 且 `s[i-len(word):i] == word`，说明可以拼接，设置 `dp[i] = True`。最终返回 `dp[len(s)]`。

复杂度分析：时间复杂度： $O(n * m * k)$ ，其中 n 是字符串长度， m 是字典大小， k 是字典中单词的平均长度。空间复杂度： $O(n)$ ，用于存储 `dp` 数组。

Python代码：

```
class Solution:
    def wordBreak(self, s: str, wordDict: list[str]) -> bool:
        word_set = set(wordDict)
        n = len(s)
        dp = [False] * (n + 1)
        dp[0] = True

        for i in range(1, n + 1):
            for word in word_set:
                if i >= len(word) and dp[i - len(word)] and s[i - len(word):i] == word:
                    dp[i] = True
                    break
        return dp[n]
```

7. 最长递增子序列（中等）

给你一个整数数组 `nums`，找到其中最严格递增子序列的长度。子序列是由数组派生而来的序列，删除（或不删除）数组中的元素而不改变其余元素的顺序。例如，`[3,6,2,7]` 是数组 `[0,3,1,6,2,2,7]` 的子序列。

示例：

```
输入: nums = [10,9,2,5,3,7,101,18]
输出: 4
解释: 最长递增子序列是 [2,3,7,101]，因此长度为 4。
```

解题思路：这是经典的最长递增子序列（LIS）问题。可以使用动态规划或者二分优化方法。动态规划思路是：用数组 `dp[i]` 表示以 `nums[i]` 结尾的最长递增子序列长度。初始化 `dp[i] = 1`，然后对于每个 `i`，遍历 `j` 从 0 到 `i-1`，如果 `nums[j] < nums[i]`，说明 `nums[i]` 可以接在 `nums[j]` 后面，更新 `dp[i] = max(dp[i], dp[j]+1)`。最终结果是 `dp` 数组中的最大值。

复杂度分析：时间复杂度： $O(n^2)$ ，其中 n 是数组长度，因为每个元素都要遍历之前的元素。空间复杂度： $O(n)$ ，用于存储 `dp` 数组。如果使用二分优化，可以把时间复杂度降到 $O(n \log n)$ 。

Python代码：

```

class Solution:
    def lengthOfLIS(self, nums: list[int]) -> int:
        if not nums:
            return 0

        dp = [1] * len(nums)

        for i in range(len(nums)):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)

```

8. 乘积最大子数组（中等）

给你一个整数数组 `nums`，请你找出数组中乘积最大的非空连续子数组（该子数组中至少包含一个数字），并返回该子数组所对应的乘积。测试用例的答案是一个 **32-位** 整数。**请注意**，一个只包含一个元素的数组的乘积是这个元素的值。

示例：

输入：nums = [2,3,-2,4]
 输出：6
 解释：子数组 [2,3] 有最大乘积 6。

解题思路：由于数组中可能包含负数，直接贪心或只记录当前最大乘积是不够的，因为负数可能将最小的乘积变为最大的。因此，需要在遍历数组时同时维护当前子数组的最大乘积 `max_prod` 和最小乘积 `min_prod`。当遇到负数时，`max_prod` 和 `min_prod` 互换。对于每个元素 `num`，更新 `max_prod = max(num, max_prod * num, min_prod * num)` 和 `min_prod = min(num, max_prod_old * num, min_prod * num)`，并用一个全局变量 `res` 记录最大值。

复杂度分析：时间复杂度：O(n)，只需一次遍历数组。空间复杂度：O(1)，只需常数个变量存储最大值和最小值。

Python代码：

```

class Solution:
    def maxProduct(self, nums: list[int]) -> int:
        if not nums:
            return 0

        max_prod = min_prod = res = nums[0]

        for num in nums[1:]:
            if num < 0:
                max_prod, min_prod = min_prod, max_prod

            max_prod = max(num, max_prod * num)
            min_prod = min(num, min_prod * num)

            res = max(res, max_prod)

```

```
return res
```

9. 分割等和子集（中等）

给你一个 **只包含正整数** 的 **非空** 数组 `nums` 。请你判断是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

示例：

```
输入: nums = [1,5,11,5]
输出: true
解释: 数组可以分割成 [1, 5, 5] 和 [11] 。
```

解题思路： 这个问题可以转化为经典的 **0-1 背包问题**：判断数组中是否存在一个子集，其和等于数组总和的一半。首先计算数组总和 `total`，如果 `total` 是奇数，则直接返回 `False`。然后使用动态规划，定义布尔数组 `dp`，其中 `dp[j]` 表示是否存在子集和为 `j`。初始化 `dp[0] = True`，然后遍历数组中的每个数 `num`，从后向前更新 `dp[j] = dp[j] or dp[j - num]`。最后检查 `dp[total // 2]` 即可。

复杂度分析： 时间复杂度： $O(n * \text{sum})$ ，其中 n 是数组长度， sum 是数组元素总和的一半。空间复杂度： $O(\text{sum})$ ，只需要一个长度为 sum 的布尔数组。

Python代码：

```
class Solution:
    def canPartition(self, nums: list[int]) -> bool:
        total = sum(nums)
        if total % 2 != 0:
            return False

        target = total // 2
        dp = [False] * (target + 1)
        dp[0] = True

        for num in nums:
            for j in range(target, num - 1, -1):
                dp[j] = dp[j] or dp[j - num]

        return dp[target]
```

10. 最长有效括号（困难）

给你一个只包含 `'('` 和 `)'` 的字符串，找出最长有效（格式正确且连续）括号子串的长度。左右括号匹配，即每个左括号都有对应的右括号将其闭合的字符串是格式正确的，比如 `"(())"`。

示例：

```
输入: s = "(())"
输出: 4
解释: 最长有效括号子串是 "(())"
```

解题思路：可以使用 **栈** 或 **动态规划** 方法解决这个问题。

1. **栈方法：**栈中保存未匹配左括号的下标，以及最后一个无效右括号的位置（作为起点）。遍历字符串时：
 - 遇到 `'('`，将其下标压入栈。
 - 遇到 `'('`，如果栈非空则弹出栈顶（匹配一个左括号），计算当前有效长度为 `i - 栈顶元素下标`；如果栈为空，则更新最后一个无效右括号下标。
 2. **动态规划方法：**定义 `dp[i]` 表示以索引 `i` 结尾的最长有效括号长度。
 - 遍历字符串，若 `s[i] == '')`：
 - 如果 `s[i-1] == '('`，则 `dp[i] = dp[i-2] + 2`
 - 如果 `s[i-1] == '')` 并且 `s[i - dp[i-1] - 1] == '('`，则 `dp[i] = dp[i-1] + 2 + dp[i - dp[i-1] - 2]`
- 最终返回 `dp` 数组的最大值。

复杂度分析：时间复杂度： $O(n)$ ，只遍历字符串一次。空间复杂度： $O(n)$ ，栈方法或 `dp` 数组需要存储最多 n 个元素。

Python代码（栈方法）：

```
class Solution:
    def longestValidParentheses(self, s: str) -> int:
        max_len = 0
        stack = [-1] # 栈底保存最后一个无效右括号下标
        for i, char in enumerate(s):
            if char == '(':
                stack.append(i)
            else:
                stack.pop()
                if not stack:
                    stack.append(i)
                else:
                    max_len = max(max_len, i - stack[-1])
        return max_len
```

十五. 多维动态规划

1. 不同路径（中等）

一个机器人位于一个 $m \times n$ 网格的左上角（起始点在下图中标记为“Start”）。机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为“Finish”）。问总共有多少条不同的路径？

示例：



输入: $m = 3, n = 7$

输出: 28

解题思路: 这是一道经典的动态规划问题。我们可以定义 $dp[i][j]$ 表示从起点 $(0,0)$ 到达格子 (i,j) 的不同路径数。由于机器人每次只能向下或者向右移动, 所以到达 (i,j) 的路径数等于到达上方格子 $(i-1,j)$ 和左方格子 $(i,j-1)$ 的路径数之和, 即 $dp[i][j] = dp[i-1][j] + dp[i][j-1]$ 。边界条件为第一行和第一列的格子只能沿一条方向移动, 因此 $dp[0][j] = 1$, $dp[i][0] = 1$ 。通过填充整个 dp 表, 可以得到最终结果 $dp[m-1][n-1]$ 。为了优化空间, 可以用一维数组滚动更新当前行的路径数。

复杂度分析: 时间复杂度为 $O(m * n)$, 因为需要填充整个 m 行 n 列的动态规划表。空间复杂度为 $O(n)$, 通过使用一维数组进行滚动更新来优化空间, 不必存储整个二维表。

Python代码:

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        dp = [1] * n # 第一行初始化为1
        for i in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j-1] # dp[j] = 上方 + 左方
        return dp[-1]
```

2. 最小路径和 (中等)

给定一个包含非负整数的 $m * n$ 网格 `grid`, 请找出一条从左上角到右下角的路径, 使得路径上的数字总和为最小。**说明:** 每次只能向下或者向右移动一步。

示例:

1	3	1
1	5	1
4	2	1

输入: `grid = [[1,3,1],[1,5,1],[4,2,1]]`

输出: 7

解释: 因为路径 $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$ 的总和最小。

解题思路: 这是一道二维网格上的动态规划问题。我们可以定义 $dp[i][j]$ 表示从左上角 $(0,0)$ 到达格子 (i,j) 的最小路径和。由于每次只能向下或向右移动, 所以 $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$ 。边界条件为第一行和第一列: 第一行只能从左向右累加, 第一列只能从上向下累加。通过顺序填充整个动态规划表, 最终 $dp[m-1][n-1]$ 就是从起点到终点的最小路径和。为了节省空间, 可以直接在原数组 `grid` 上进行就地更新, 减少额外空间开销。

复杂度分析: 时间复杂度为 $O(m * n)$, 需要遍历整个 $m * n$ 的网格。空间复杂度为 $O(1)$, 如果直接在原网格 `grid` 上进行动态规划更新, 不使用额外的 dp 表; 若使用额外的二维 dp 表, 则空间复杂度为 $O(m * n)$ 。

Python代码:

```

class Solution:
    def minPathSum(self, grid: list[list[int]]) -> int:
        m, n = len(grid), len(grid[0])
        # 更新第一行
        for j in range(1, n):
            grid[0][j] += grid[0][j-1]
        # 更新第一列
        for i in range(1, m):
            grid[i][0] += grid[i-1][0]
        # 更新其他格子
        for i in range(1, m):
            for j in range(1, n):
                grid[i][j] += min(grid[i-1][j], grid[i][j-1])
        return grid[-1][-1]

```

3. 最长回文子串 (中等)

给你一个字符串 `s`，找到 `s` 中最长的回文子串。

示例：

输入: `s = "babad"`
 输出: `"bab"`
 解释: `"aba"` 同样是符合题意的答案。

解题思路：本题可以使用“中心扩展法”解决。一个回文串的中心可以是一个字符（奇数长度回文）或者两个相邻字符之间（偶数长度回文）。我们对每个字符都尝试作为中心向两边扩展，找到以它为中心的最长回文子串。记录最长回文子串的起始位置和长度，最终返回对应的子串即可。该方法无需使用额外的动态规划数组，代码简洁且效率较高。

复杂度分析：时间复杂度为 $O(n^2)$ ，因为每个字符都可能扩展到整个字符串的长度。空间复杂度为 $O(1)$ ，仅使用常数额外空间来记录最长回文子串的位置和长度。

Python代码：

```

class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ""

        start, max_len = 0, 0

        def expand(l: int, r: int):
            while l >= 0 and r < len(s) and s[l] == s[r]:
                l -= 1
                r += 1
            return l + 1, r - 1

        for i in range(len(s)):
            # 奇数长度回文
            l1, r1 = expand(i, i)
            # 偶数长度回文
            l2, r2 = expand(i, i + 1)

```

```

        if r1 - l1 + 1 > max_len:
            start, max_len = l1, r1 - l1 + 1
        if r2 - l2 + 1 > max_len:
            start, max_len = l2, r2 - l2 + 1

    return s[start:start + max_len]

```

4. 最长公共子序列（中等）

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长 **公共子序列** 的长度。如果不存在 **公共子序列**，返回 `0`。一个字符串的 **子序列** 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。例如，`"ace"` 是 `"abcde"` 的子序列，但 `"aec"` 不是 `"abcde"` 的子序列。两个字符串的 **公共子序列** 是这两个字符串所共同拥有的子序列。

示例：

输入: `text1 = "abcde"`, `text2 = "ace"`
 输出: `3`
 解释: 最长公共子序列是 `"ace"`，它的长度为 `3`。

解题思路： 本题可以使用动态规划解决。定义二维数组 `dp`，其中 `dp[i][j]` 表示字符串 `text1[0..i-1]` 与 `text2[0..j-1]` 的最长公共子序列长度。状态转移方程如下：如果 `text1[i-1] == text2[j-1]`，说明当前字符可以作为公共子序列的一部分，`dp[i][j] = dp[i-1][j-1] + 1`；否则，`dp[i][j] = max(dp[i-1][j], dp[i][j-1])`，表示要么跳过 `text1[i-1]`，要么跳过 `text2[j-1]`，取最大值。最终 `dp[m][n]` 就是最长公共子序列的长度。

复杂度分析： 时间复杂度为 $O(mn)$ ，其中 m 和 n 分别为 `text1` 和 `text2` 的长度。空间复杂度为 $O(mn)$ ，用于存储动态规划数组 `dp`。可以通过优化，使用两行滚动数组将空间复杂度降到 $O(\min(m,n))$ 。

Python代码：

```

class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m, n = len(text1), len(text2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + 1
                else:
                    dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

        return dp[m][n]

```

5. 编辑距离 (中等)

给你两个单词 `word1` 和 `word2`，请返回将 `word1` 转换成 `word2` 所使用的最少操作数。你可以对一个单词进行如下三种操作：插入一个字符。删除一个字符。替换一个字符。

示例：

```
输入: word1 = "horse", word2 = "ros"
输出: 3
解释:
horse -> rorse (将 'h' 替换为 'r')
rorse -> rose (删除 'r')
rose -> ros (删除 'e')
```

解题思路：本题可以使用动态规划解决。定义二维数组 `dp`，其中 `dp[i][j]` 表示将 `word1[0..i-1]` 转换为 `word2[0..j-1]` 所需的最少操作次数。状态转移方程如下：

1. 如果 `word1[i-1] == word2[j-1]`，则无需操作，`dp[i][j] = dp[i-1][j-1]`。
2. 否则，可以选择三种操作之一：
 - 插入一个字符：`dp[i][j-1] + 1`
 - 删除一个字符：`dp[i-1][j] + 1`
 - 替换一个字符：`dp[i-1][j-1] + 1`取三者的最小值作为 `dp[i][j]`。最终 `dp[m][n]` 就是将 `word1` 转换为 `word2` 的最少操作数。

复杂度分析：时间复杂度为 $O(mn)$ ，其中 m 和 n 分别是 `word1` 和 `word2` 的长度，需要遍历二维数组。空间复杂度为 $O(mn)$ ，用于存储动态规划数组 `dp`。可通过滚动数组优化为 $O(\min(m,n))$ 。

Python代码：

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        # 初始化边界情况
        for i in range(m + 1):
            dp[i][0] = i # 删除操作
        for j in range(n + 1):
            dp[0][j] = j # 插入操作

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = min(
                        dp[i - 1][j] + 1, # 删除
                        dp[i][j - 1] + 1, # 插入
                        dp[i - 1][j - 1] + 1 # 替换
                    )

        return dp[m][n]
```


十六. 技巧

1. 只出现一次的数字（简单）

给你一个 **非空** 整数数组 `nums`，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。你必须设计并实现线性时间复杂度的算法来解决此问题，且该算法只使用常量额外空间。

示例：

```
输入: nums = [2,2,1]
输出: 1
```

解题思路：本题可以利用 **异或运算** 的性质解决：

1. 异或满足交换律和结合律，即 $a \oplus b \oplus a = b$ 。
2. 将数组中所有元素依次进行异或操作，相同的数字会互相抵消为 0，剩下的就是只出现一次的数字。

由于只需遍历数组一次并使用一个变量存储结果，所以空间复杂度为 $O(1)$ ，时间复杂度为 $O(n)$ 。

复杂度分析：时间复杂度： $O(n)$ ，只需要对数组进行一次遍历。空间复杂度： $O(1)$ ，只使用了一个额外变量来存储异或结果。

Python代码：

```
class Solution:
    def singleNumber(self, nums: list[int]) -> int:
        result = 0
        for num in nums:
            result ^= num
        return result
```

2. 多数元素（简单）

给定一个大小为 `n` 的数组 `nums`，返回其中的多数元素。多数元素是指在数组中出现次数 **大于** $\lfloor n/2 \rfloor$ 的元素。你可以假设数组是非空的，并且给定的数组总是存在多数元素。

示例：

```
输入: nums = [3,2,3]
输出: 3
```

解题思路：本题可以使用 **摩尔投票算法 (Boyer-Moore Voting Algorithm)** 高效解决。算法的核心思想是：如果我们每次从数组中删除一个不同的多数元素和非多数元素配对，最后剩下的一定是多数元素。具体实现为维护一个候选元素和计数器：遍历数组，如果计数器为 0，就将当前元素设为候选；如果当前元素等于候选，计数器加 1，否则计数器减 1。遍历结束时，候选元素即为多数元素。

复杂度分析：时间复杂度： $O(n)$ ，只需要对数组遍历一次。空间复杂度： $O(1)$ ，只使用了两个变量存储候选和计数。

Python代码:

```
class Solution:
    def majorityElement(self, nums: list[int]) -> int:
        candidate = None
        count = 0
        for num in nums:
            if count == 0:
                candidate = num
                count = 1
            elif num == candidate:
                count += 1
            else:
                count -= 1
        return candidate
```

3. 颜色分类 (中等)

给定一个包含红色、白色和蓝色、共 n 个元素的数组 `nums`，原地对它们进行排序，使得相同颜色的元素相邻，并按照红色、白色、蓝色顺序排列。我们使用整数 `0`、`1` 和 `2` 分别表示红色、白色和蓝色。必须在不使用库内置的 `sort` 函数的情况下解决这个问题。

示例:

```
输入: nums = [2,0,2,1,1,0]
输出: [0,0,1,1,2,2]
```

解题思路: 本题可以使用 **双指针/荷兰国旗问题 (Dutch National Flag Problem)** 方法高效解决。思想是维护三个区域: 左边是 0 (红色)，中间是 1 (白色)，右边是 2 (蓝色)。定义三个指针: `p0` 指向下一个 0 的位置, `p2` 指向下一个 2 的位置, `current` 指向当前遍历的位置。遍历数组时, 如果 `nums[current]` 为 0, 就与 `p0` 位置交换, 并同时移动 `p0` 和 `current`; 如果为 2, 就与 `p2` 位置交换, 并移动 `p2` (注意此时 `current` 不动, 需要再次检查交换后的值); 如果为 1, 则直接移动 `current`。遍历完成后数组即按红白蓝顺序排序。

复杂度分析: 时间复杂度: $O(n)$, 每个元素最多被访问两次。空间复杂度: $O(1)$, 只使用了常数个指针变量。

Python代码:

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        p0, current = 0, 0
        p2 = len(nums) - 1

        while current <= p2:
            if nums[current] == 0:
                nums[p0], nums[current] = nums[current], nums[p0]
                p0 += 1
                current += 1
            elif nums[current] == 2:
                nums[p2], nums[current] = nums[current], nums[p2]
                p2 -= 1
```

```
else:
    current += 1
```

4. 下一个排列（中等）

整数数组的一个 **排列** 就是将其所有成员以序列或线性顺序排列。

- 例如，`arr = [1,2,3]`，以下这些都可以视作 `arr` 的排列：`[1,2,3]`、`[1,3,2]`、`[3,1,2]`、`[2,3,1]`。

整数数组的 **下一个排列** 是指其整数的下一个字典序更大的排列。更正式地，如果数组的所有排列根据其字典顺序从小到大排列在一个容器中，那么数组的 **下一个排列** 就是在这个有序容器中排在它后面的那个排列。如果不存在下一个更大的排列，那么这个数组必须重排为字典序最小的排列（即，其元素按升序排列）。

- 例如，`arr = [1,2,3]` 的下一个排列是 `[1,3,2]`。
- 类似地，`arr = [2,3,1]` 的下一个排列是 `[3,1,2]`。
- 而 `arr = [3,2,1]` 的下一个排列是 `[1,2,3]`，因为 `[3,2,1]` 不存在一个字典序更大的排列。

给你一个整数数组 `nums`，找出 `nums` 的下一个排列。必须原地修改，只允许使用额外常数空间。

示例：

```
输入: nums = [1,2,3]
输出: [1,3,2]
```

解题思路： 本题使用 **字典序排列算法**。首先从后向前找到第一个满足 `nums[i] < nums[i+1]` 的位置 `i`，这是第一个需要改变的位置（从后向前找到的第一个升序对），说明从 `i+1` 到末尾是降序排列的。然后在后半段找到第一个大于 `nums[i]` 的元素 `nums[j]`，交换 `nums[i]` 和 `nums[j]`，这样保证改变后的排列是比原排列略大的最小排列。最后将 `i+1` 到末尾的序列反转（因为原本是降序，需要变成升序），就得到下一个字典序排列。如果整个数组是降序排列，则直接反转整个数组得到最小排列。

复杂度分析： 时间复杂度：O(n)，只需遍历数组两到三次即可完成操作。空间复杂度：O(1)，原地修改，只使用常数空间。

Python代码：

```
class Solution:
    def nextPermutation(self, nums: list[int]) -> None:
        n = len(nums)
        i = n - 2
        # 找到第一个下降的位置
        while i >= 0 and nums[i] >= nums[i + 1]:
            i -= 1
        if i >= 0:
            # 找到后面比 nums[i] 大的最小值
            j = n - 1
            while nums[j] <= nums[i]:
                j -= 1
            nums[i], nums[j] = nums[j], nums[i]
        # 反转 i+1 到末尾
        left, right = i + 1, n - 1
        while left < right:
```

```
nums[left], nums[right] = nums[right], nums[left]
left += 1
right -= 1
```

5. 寻找重复数 (中等)

给定一个包含 $n + 1$ 个整数的数组 `nums`，其数字都在 $[1, n]$ 范围内（包括 `1` 和 `n`），可知至少存在一个重复的整数。假设 `nums` 只有 **一个重复的整数**，返回 **这个重复的数**。你设计的解决方案必须 **不修改** 数组 `nums` 且只用常量级 $O(1)$ 的额外空间。

示例：

```
输入: nums = [1,3,4,2,2]
输出: 2
```

解题思路：本题可以使用 **快慢指针 (Floyd 判圈算法)** 来寻找重复数，将数组看作链表，每个元素的值指向下一个节点的索引：`next = nums[current]`。由于数组长度为 $n+1$ 且数字范围为 $[1, n]$ ，必然存在环（重复数字导致多个索引指向同一个节点）。首先使用快慢指针找到环的相遇点，然后将其中一个指针重置到起点，两个指针同步移动，相遇点即为重复的数字。这种方法不修改数组，且只用常数额外空间。

复杂度分析：时间复杂度： $O(n)$ ，快慢指针最多遍历 $2n$ 次即可找到环和重复数。空间复杂度： $O(1)$ ，只使用两个指针，无额外数组或哈希表。

Python代码：

```
class Solution:
    def findDuplicate(self, nums: list[int]) -> int:
        # 快慢指针初始化
        slow = fast = nums[0]
        # 第一次相遇，找到环
        while True:
            slow = nums[slow]
            fast = nums[nums[fast]]
            if slow == fast:
                break
        # 找到入口
        slow = nums[0]
        while slow != fast:
            slow = nums[slow]
            fast = nums[fast]
        return slow
```